

Helix: Scalable Multi-Party Machine Learning Inference against Malicious Adversaries

Yansong Zhang, Xiaojun Chen, Qinghui Zhang, Xudong Chen, and Ye Dong

Abstract—With the growing emphasis on data privacy, secure multi-party computation has garnered significant attention for its strong security guarantees in developing privacy-preserving machine learning (PPML) schemes. However, only a few works address scenarios with a large number of participants. The state of the art by Liu *et al.* (LXY24, USENIX Security’24) first achieves a practical PPML protocol for up to 63 parties but is constrained to semi-honest security. Although naive extensions to the malicious setting are feasible, they would introduce significant overhead in verifying the correctness of multiplications. In this paper, we design Helix, a scalable framework for maliciously secure PPML in the honest majority setting, aiming to enhance both the scalability and practicality of maliciously secure protocols. In particular, we first report a privacy leakage issue in LXY24 during prefix OR operations and introduce a round-optimized alternative based on a single-round vectorized four-input multiplication protocol. To mitigate the verification burden, we propose a set of lightweight compression protocols by exploiting *reusability* properties within the computation process, and seamlessly integrate them into existing verification techniques. Building on these enhancements, we further construct a practically-efficient and general n -party computation protocol that serves as the cryptographic foundation for advanced PPML schemes. As a result, Helix achieves efficiency comparable to semi-honest frameworks. For instance, in 63-party neural network inference, Helix is only $1.9\times$ ($1.1\times$) slower in the online phase and $1.2\times$ ($1.1\times$) slower in preprocessing under LAN (WAN), compared to LXY24, in the best case.

Index Terms—secure multi-party computation, malicious security, honest majority, privacy-preserving machine learning

I. INTRODUCTION

MACHINE learning (ML) is increasingly applied across diverse domains, including medicine, finance, and recommendation systems. However, this widespread success has raised significant privacy concerns regarding both models and personal data. As such, privacy-preserving techniques should be employed to ensure the privacy of the data used in machine-learning-as-a-service.

Secure multi-party computation (MPC) [1] is a notable approach for enabling privacy-preserving machine learning (PPML). MPC allows n parties to collaboratively compute a function over their private inputs while ensuring input privacy and output correctness. Nowadays, MPC-based PPML has made significant progress, particularly under the semi-honest

setting [2], [3], [4], where adversaries honestly follow the protocol but try to learn secret values. Nevertheless, in real-world scenarios involving large numbers of participants, such as federated learning, expecting that all individual parties are semi-honest is overly strong and impractical. Therefore, there is an urgent need for scalable protocols that support efficient PPML with many parties in the malicious setting, where adversaries can arbitrarily deviate from the protocol.

However, existing malicious PPML protocols primarily focus on 2-4 parties [5], [6], [7], [8], [9], [10], [11], where one party is corrupt. Although MPC protocols in the dishonest majority setting theoretically support an arbitrary number of parties, current PPML implementations [12], [13], [14] demonstrate substantial latency and overhead even in 2-party computations. In the most recent work, Liu *et al.* [15] utilize Shamir secret sharing [16] to design scalable PPML protocols under the semi-honest secure honest majority model. Their approach enables efficient PPML inference with up to 63 parties. For ease of reference, we denote this work [15] as “LXY24”. Trivially, the protocols in LXY24 can be adapted to provide malicious security using standard theoretical approaches [17], [18], [19], [20], with the primary bottleneck being the verification of multiplication operations. Nevertheless, due to the inherent design limitations of the protocols in LXY24 and the large scale of applications, the naive extensions face two critical issues as follows.

First, a privacy leakage issue exists in LXY24. The comparison protocol in LXY24 achieves both constant rounds and low communication costs without requiring a gap between the domains of shares and secrets. However, rigorous security analysis reveals a privacy leakage issue in the prefix OR protocol, which forms the core component and the primary bottleneck of the comparison protocol. This vulnerability can expose all input values of the prefix OR operation, thereby compromising the security of the comparison protocol and subsequent non-linear function computations. Furthermore, it indicates that the n -party comparison protocols over fields, with no gap requirement, still encounter a trade-off¹ between rounds and communication costs.

Second, the verification step exhibits performance limitations. In large-scale applications, the number of verified multiplication triples N is considerable. In such cases, the multiplication verification method in [17] incurs significant communication overhead due to its complexity linear in N . Protocols that achieve sublinear communication [18], [19], [20] have higher computational complexity, which also grows

Yansong Zhang, Xiaojun Chen, Qinghui Zhang and Xudong Chen are with the Institute of Information Engineering, State Key Laboratory of Cyberspace Security Defense, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: zhangyansong@iie.ac.cn; chenxiaojun@iie.ac.cn; zhangqinghui@iie.ac.cn; chenxudong@iie.ac.cn).

Ye Dong is with the National University of Singapore, Singapore 119077 (e-mail: dongye@nus.edu.sg)

¹For further details, please refer to Section VIII.

linearly with N . As a result, in practical scenarios with large N , such as neural network (NN) inference, existing verification methods suffer from either high communication costs or great computational burden, which causes a significant performance gap between malicious and semi-honest protocols. For instance, in our experiments, the state-of-the-art malicious extension [18] for LXY24 results in a $3.6\times$ to $6.1\times$ increase in online NN inference time under LAN compared to the original LXY24.

To address the first issue, we employ the DN [21] protocol, which supports secure multiplication even when the input secret values contain zeros (the root cause of the aforementioned privacy issue), to securely realize the prefix multiplication operation. In this context, prefix multiplication effectively resolves the dual problem of prefix OR. Furthermore, to optimize the round complexity, we extend the vectorized two-layer (three-input) DN multiplication in LXY24 to a four-input version by leveraging the Beaver triples [22], without increasing online communication. The three-input and four-input DN multiplication protocols enable prefix multiplications with 2-4 inputs to be completed in only one online round. Ultimately, inspired by [23], [24], [25], we construct a round-efficient prefix multiplication protocol by grouping computations in sets of four and utilizing these efficient building blocks. Theoretically, our improved prefix OR protocol achieves a better trade-off between online rounds and communication complexity, which requires only $\log_4 \ell$ rounds and $\frac{\ell-4}{3} + \frac{3}{2}\ell \log_4 \ell$ (i.e., $O(\ell \log_4 \ell)$) field elements per party in total.

To address the second issue, we primarily utilize the *random linear combination* technique [26], [27], which introduces randomness to reduce the verification scale while ensuring correctness with overwhelming probability. First, we employ the multiplication verification protocol in [18] to achieve sub-linear communication and tailor it to support machine learning applications based on the linearity of Shamir secret sharing and the random linear combination technique. In addition, to further reduce computational complexity, our key insight is that *large numbers of multiplication operations, such as those in matrix multiplication, share a common multiplier or involve multipliers related through affine transformations*. Thus, rather than verifying such triples independently, we propose a lightweight compression protocol to consolidate them into a single triple with random linear combinations. In this way, we can significantly reduce N , thereby improving verification efficiency and enhancing practicality. Specifically, for the matrix multiplication of dimensions $m \times n$ and $n \times o$, the verification count is reduced to $\frac{1}{mo}$ of the original requirement. As for non-linear functions, the number of triples requiring verification is reduced from $O(\ell \log_4 \ell)$ to $O(\ell)$. Finally, we seamlessly integrate the above protocols to propose a maliciously secure general-purpose MPC protocol for efficiently computing arbitrary arithmetic circuits, with a particular emphasis on machine learning applications.

Building upon the above techniques, we further enhance the protocols in LXY24 to efficiently achieve malicious security and introduce Helix, a scalable framework designed for multi-party machine learning inference with malicious security in the honest majority setting.

In brief, we summarize our main contributions as follows.

- We identify and address the privacy leakage issue in the prefix OR protocol proposed in LXY24. Specifically, we adopt a four-input version of the vectorized two-layer multiplication protocol in LXY24, without increasing online communication. We further combine these two protocols to design a secure and optimized prefix OR protocol, achieving the online complexity of $\log_4 \ell$ rounds with $O(\ell \log_4 \ell)$ field elements per party.
- We propose a lightweight multiplication triple compression protocol by leveraging the *reusability* properties, significantly reducing the number of verified multiplications by up to 90% in NN inference. Notably, this new protocol is compatible with arbitrary verification techniques, which is of independent interest.
- We extend the multiplication verification protocol in [18] to support machine learning applications without introducing additional communication rounds. This is further integrated with our compression protocols to yield a maliciously-secure and general-purpose MPC protocol, which serves as the cryptographic foundation for constructing PPML applications.
- We implement Helix² and evaluate its performance in secure NN inference with varying numbers of parties, ranging from 3 to 63. In the best case, Helix is $1.9\times$ ($1.1\times$) slower in the online phase and $1.2\times$ ($1.1\times$) slower in preprocessing under LAN (WAN) compared to the state-of-the-art semi-honest secure protocol, LXY24 [15]. Additionally, compared to the naive approach that directly employs the protocol from [18] for malicious security, Helix achieves up to $2.6\times$ faster online NN inference in LAN.

II. PRELIMINARIES

Notations: Let $P_1, \dots, P_n$ be the n parties to do the secure computation. We denote scalar, vector, and matrix by lowercase letter x , lowercase bold letter $\mathbf{x}$, and uppercase bold letter $\mathbf{X}$, respectively. Let $\mathbf{X}(i, j)$ denote the element at the i -th row and j -th column in matrix $\mathbf{X}$, and $\mathbf{x}(i)$ the i -th element in vector $\mathbf{x}$. Denote the functionality and protocol used in the semi-honest setting by $\mathcal{F}^{\text{sh}}$ and Π^{sh} , respectively. Similarly, $\mathcal{F}^{\text{mal}}$ ($\mathcal{F}$ for simplicity) and Π^{mal} (Π for simplicity) are employed in the malicious setting. We use $[a, b]$ to denote the set $\{a, a+1, \dots, b\}$.

A. An Overview of LXY24

LXY24 [15] designs a suite of secure multi-party computation protocols for privacy-preserving inference with semi-honest security in the honest majority setting. In this section, we briefly recall the secret-sharing scheme of LXY24, which serves as the foundation for the inference protocols.

Shamir Secret Sharing. LXY24 is based on Shamir's (t, n) -threshold scheme [16], where n is the number of parties, t is the number of corrupted parties, and $n \geq 2t + 1$. For the rest of the paper, we assume maximal corruption in this

²We will release the code upon acceptance.

setting, and thus $n = 2t + 1$. To share a secret $x \in \mathbb{F}_p$ with degree t , a uniformly random polynomial $f(X)$ of degree t is chosen under the constraint that $f(0) = x$. Each party P_i holds the share $f(\alpha_i)$, where $\alpha_i \in \mathbb{F}_p$ is the unique identifier for P_i . For convenience, we set $\alpha_i = i$. In LXY24, the finite field $\mathbb{F}_p$ is defined over a fixed Mersenne prime $p = 2^\ell - 1$, with examples including $\ell = 31, 61$. We denote the degree- t sharing as $[\cdot]$ -sharing. Multiplying two degree- t sharings yields a degree- $2t$ sharing, denoted by $[\![\cdot]\!]$ -sharing, with no communication. Additionally, any linear function or addition by a constant can be performed locally. Thus, for simplicity, we write $[(a + b) \bmod p] = [a + b] = [a] + [b]$ and $[(a \cdot b) \bmod p] = [ab] = [a] \cdot [b]$ for shares $[a]$ and $[b]$.

Sharing and Reconstruction. $\Pi_{\text{Share}}^{\text{sh}}$ works as follows: the dealer chooses a random polynomial $f(X)$ and sends each P_i the point $f(i)$. To reveal a secret $[x]$, at least t parties send their shares to P_{king} , who then reconstructs x and sends it back to the other parties. We write $x \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([x])$ and $x \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([\![x]\!])$ for the revealing of degree- t and degree- $2t$ sharings³, respectively. In LXY24, the round complexity of a protocol is measured by the number of rounds of parallel invocations of $\Pi_{\text{Reveal}}^{\text{sh}}$.

Random Sharings Generation. Based on the Vandermonde matrix and sharing protocol $\Pi_{\text{Share}}^{\text{sh}}$, random degree- t sharings and random double sharings can be efficiently generated [21]. We define the generation protocols as $[r] \leftarrow \Pi_{\text{Rand}}^{\text{sh}}$ and $([r], [\![r]\!]) \leftarrow \Pi_{\text{DoubleRand}}^{\text{sh}}$, respectively.

DN Multiplication. The multiplication between two degree- t sharings can be realized by DN multiplication [21] with the use of $([r], [\![r]\!])$. We write the DN multiplication protocol as $[xy] \leftarrow \Pi_{\text{Mult}}^{\text{sh}}([x], [y])$. In essence, the DN multiplication protocol performs a degree reduction from $[\![xy]\!]$ to $[xy]$ as follows:

1. All parties locally compute $[\![xy + r]\!] = [\![xy]\!] + [\![r]\!] = [x] \cdot [y] + [r]$ and send their shares to P_{king} .
2. P_{king} reconstructs the value $xy + r$ and sends it back to other parties.
3. All parties locally compute $[xy] = (xy + r) - [r]$,

For two vectors of degree- t sharings $[x]$ and $[y]$, to compute $[x \cdot y]$, we can first compute $[\![x \cdot y]\!] = [x] \cdot [y]$ and then apply the same approach as the DN multiplication protocol to compute $[x \cdot y]$ from $[\![x \cdot y]\!]$. We define the protocol as $\Pi_{\text{InnerProd}}^{\text{sh}}$.

Multiplication with Public Output. For multiplication with public output, the protocol leverages the pseudo-random sharing of zero (PRZS [28]) technique. Specifically, all parties first generate $[0]$ without interactions by using PRZS. The final output is then computed as $xy \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([x] \cdot [y] + [0])$. We write this protocol as $xy \leftarrow \Pi_{\text{MultPub}}^{\text{sh}}([x], [y])$.

2L-DN Multiplication. The consecutive multiplication of three degree- t sharings can be achieved in a single on-line round through the vectorized two-layer DN multiplication protocol [15], [29], denoted as $\{xyz_i\}_{i=1}^m \leftarrow \Pi_{\text{2L-DN}}^{\text{sh}}([x], [y], \{[z_i]\}_{i=1}^m)$. In the preprocessing phase, parties generate $m + 1$ pairs of double sharings $([r], [\![r]\!])$ and $([\![r_i]\!], [\![r_i]\!])\}_{i=1}^m$. In the online phase, all parties execute the

following steps: 1) Compute $[\![u]\!] = [x] \cdot [y] + [r]$ and $[\![u_i]\!] = [r] \cdot [-z_i] + [\![r_i]\!]$. 2) Reveal $u \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([\![u]\!])$ and $u_i \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([\![u_i]\!])$. 3) Locally compute $[xyz_i] = u \cdot [z_i] + u_i - [r_i]$ for $1 \leq i \leq m$.

Optimization using PRG. The communication complexity of the above protocols can be further reduced by utilizing pseudo-random generators (PRG) [29], [17]. Using PRG, distributing a degree- t sharing and a degree- $2t$ sharing requires only $n - t - 1$ and $n - 2t - 1$ field elements of communication, respectively. Thus, when $n = 2t + 1$, both a random sharing and a pair of double sharing can be generated with a communication cost of 1 field element per party. In the complexity analysis across this work, we adopt the costs from the PRG-optimized version.

B. Useful Techniques in the Malicious Setting

Revealing Degree- t Sharings with Correctness Check. In the malicious setting, revealing a degree- t sharing requires each party to distribute its share to all other parties. Then the correctness of the revealed sharings can be checked as follows: each party P_i uses any of the $t + 1$ shares to compute the unique degree- t polynomial, and checks that all other shares lie on the same polynomial. If not, then it outputs $\perp$ and aborts. We write the protocol for revealing degree- t sharings with correctness check as $\Pi_{\text{Reveal-Check}}$.

Random Coins Generation. There are two methods to implement the protocol Π_{Coin} in the malicious setting. First, as described in [17], [18], the parties invoke $\Pi_{\text{Rand}}^{\text{sh}}$ to generate a random sharing and then reveal the result with the use of $\Pi_{\text{Reveal-Check}}$. The second method, as described in [30], requires each party to sample a seed and broadcast its commitment. The parties then reveal all commitments and XOR the seeds. Finally, random field elements can be generated by using PRG.

Batch Correctness Check of Shares. In [17], the correctness of m degree- t sharings can be checked in batch utilizing $\Pi_{\text{Reveal-Check}}$. For $[x_1], \dots, [x_m]$, the parties first generate m non-zero random field elements $\{\alpha_i\}_{i=1}^m$ using Π_{Coin} and a random sharing $[r]$ using $\Pi_{\text{Rand}}^{\text{sh}}$. Then compute $[v] = \sum_{i=1}^m \alpha_i \cdot [x_i] + [r]$ locally. Finally, each party broadcasts its share of $[v]$ and checks the correctness using $\Pi_{\text{Reveal-Check}}$. If no abort, then the parties accept all input shares. We write the above process as $\Pi_{\text{ShareCheck}}$.

By executing $\Pi_{\text{ShareCheck}}$ on degree- t sharings generated by $\Pi_{\text{Share}}^{\text{sh}}$, $\Pi_{\text{Rand}}^{\text{sh}}$, and $\Pi_{\text{DoubleRand}}^{\text{sh}}$, we can straightforwardly obtain maliciously secure versions of the protocols: Π_{Share} , Π_{Rand} , and $\Pi_{\text{DoubleRand}}$, which ensure that the output shares are valid sharings of some values.

Multiplication with Security Up to Additive Attacks. To achieve security up to additive attacks in the malicious setting, [17] introduces the following small modifications to the DN multiplication protocol: i) Random double sharings must be generated using $\Pi_{\text{DoubleRand}}$, ensuring that $[r]$ is a correct sharing and that $(xy + r) - [r]$ is correct as well. ii) To prevent a corrupt P_{king} from sending different values (e.g., $xy + r$) to the parties, an additional step is introduced where parties compare the hash of their view, i.e., the values received from P_{king} . We write the modified DN multiplication as Π_{Mult} . And it is easy to obtain $\Pi_{\text{InnerProd}}$ with similar modifications. Note that these

³Similarly, for degree- $2t$ sharings, at least $2t$ parties send their shares to P_{king} .

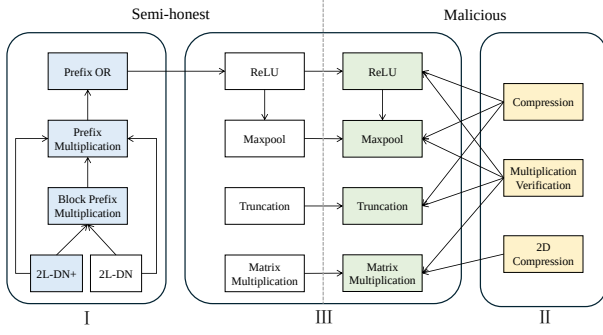


Fig. 1. Dependency of the protocols considered in Helix. An arrow pointing from A to B means that protocol B requires protocol A. White-bordered boxes represent the protocols in LXY24, while colored boxes indicate the newly proposed protocols in this work.

two protocols do not guarantee that the output is the correct multiplication result of the inputs. Rather, they only ensure the output shares constitute a valid sharing of some value.

III. DESIGN OF HELIX

A. Architecture

We consider a scenario where n (≥ 3) parties $\{P_1, \dots, P_n\}$ collaboratively perform machine learning inference on their private raw data. Our Helix contains a series of secure n -party protocols to achieve this goal. Fig. 1 shows the hierarchy among these protocols, which can be divided into three parts:

- **I** (Section IV): We propose a novel prefix OR protocol to replace the insecure variant in LXY24, which constitutes the core of nonlinear functions.
- **II** (Section V): We introduce lightweight multiplication triple compression protocols to enhance the efficiency of batch multiplication verification.
- **III** (Section VI): We integrate our improved verification method into semi-honest protocols and build maliciously secure blocks for widely used NN operators, including matrix multiplication, truncation, and ReLU.

B. Security Model

In this work, we consider security against static malicious adversaries in the honest-majority setting. Specifically, an adversary corrupts $t < n/2$ parties at the beginning of the protocol execution and can deviate from the protocol specification arbitrarily. The goal of our security model is to prove that the malicious adversary has no impact on the computational result or the privacy of the sensitive inputs. Formally, we model and prove the security of our protocols under the universal composition (UC) framework [31]. Due to space constraints, we formally describe the functionalities mentioned in this work in Appendix A of supplementary materials, available online.

IV. IMPROVED BITWISE PRIMITIVES

In this section, we first analyze the potential privacy leakage issue in LXY24 (Section IV-A). Next, we propose a semi-honest secure and round-optimized alternative solution (Section IV-B, IV-C, IV-D). In Section VI, we apply these protocols to the malicious setting.

A. The Security Issue in LXY24

Protocol 1: $\Pi_{\text{PreOR}}^{\text{sh}}$

Input: $[a_1], [a_2], \dots, [a_\ell]$ where $a_i \in \{0, 1\}$.

Output: $[b_1], [b_2], \dots, [b_\ell]$ where $b_i = \bigvee_{j=1}^i a_j$.

1. For $1 \leq i \leq \ell$, compute $[\bar{a}_i] = 1 - [a_i]$.
2. $\{[\prod_{j=1}^i \bar{a}_j]\}_{i=1}^\ell \leftarrow \mathcal{F}_{\text{PreMult}}^{\text{sh}}(\{[\bar{a}_i]\}_{i=1}^\ell)$.
3. For $1 \leq i \leq \ell$, output $[b_i] = 1 - [\prod_{j=1}^i \bar{a}_j]$.

In LXY24, the prefix OR operation is calculated by solving its dual problem $\mathcal{F}_{\text{PreMult}}^{\text{sh}}$, as illustrated in Protocol 1. The functionality $\mathcal{F}_{\text{PreMult}}^{\text{sh}}$ in LXY24 is implemented based on the constant-round prefix multiplication techniques proposed in [32], [33], [34], [35]. Specifically, in the preprocessing phase, parties prepare ℓ pairs of correlated random sharings $([r_i], [r'_i])$ where $r'_1 = r_1^{-1}$ and $r'_i = r_{i-1}r_i^{-1}$ for $i > 1$. In the online phase, for each $i \in [1, \ell]$, the parties compute and reveal $c_i \leftarrow \Pi_{\text{MultPub}}^{\text{sh}}([\bar{a}_i], [r'_i])$ and then locally compute $[r_i] \cdot \prod_{j=1}^i c_j$, which yields the prefix products.

However, we observe that this instantiation fails to UC-securely realize the functionality $\mathcal{F}_{\text{PreMult}}^{\text{sh}}$ as defined in LXY24, because the public values $\{c_i\}_{i=1}^\ell$ are not uniformly distributed over $\mathbb{F}_p$ when handling binary inputs, leading to a potential privacy leakage issue⁴.

Now we demonstrate that this issue will lead to the leakage of all private inputs to the prefix OR protocol. Note that all the correlated random elements $\{(r_i, r'_i)\}_{i=1}^\ell$ are non-zero, as the computation of r'_i necessitates a multiplicative inverse. Moreover, each element in $\{\bar{a}_i\}_{i=1}^\ell$ is either 0 or 1 in the prefix OR protocol. As a consequence, if $\bar{a}_i = 0$, the corresponding public value c_i is also 0, whereas if $\bar{a}_i = 1$, c_i becomes non-zero. Hence, an adversary can deduce the private values $\{\bar{a}_i\}_{i=1}^\ell$ by observing if the public values $\{c_i\}_{i=1}^\ell$ are equal to 0.

Our solution: We employ the DN protocol to present an alternative solution since it securely computes the multiplication of binary inputs by masking the opened values with uniformly random values, thereby avoiding the above privacy leakage issue. To optimize the round complexity, we further design a four-input multiplication protocol in Section IV-B and block prefix multiplication protocols in Section IV-C, both based on DN protocol. Finally, we construct a round-efficient and secure prefix multiplication protocol in Section IV-D.

B. Vectorized Two-Layer DN Multiplication: An Extension

We extend $\Pi_{\text{2L-DN}}^{\text{sh}}$ to a four-input protocol using Beaver triples [22], without increasing online communication cost [29]. For ease of description, we consider the general case where the last two multipliers are vectors of equal length. That is given $[x], [y], \{[z_i]\}_{i=1}^m$ and $\{[w_i]\}_{i=1}^m$, the goal is to compute $\{[xyz_iw_i]\}_{i=1}^m$. We begin by computing the standard DN multiplication $[u] = [x] \cdot [y] + [r]$ and $[u_i] = [z_i] \cdot [w_i] + [r_i]$, where $([r], [r_i])$ and $([r_i], [r_i])$ are random double sharings.

⁴Previous works [32], [33], [34], [35] handle inputs over $\mathbb{F}_p^*$ and thus they do not encounter such issue.

Since $[r]$ and $[r_i]$ are generated in the preprocessing phase, $[c_i] = [r \cdot r_i]$ can be precomputed, which forms a Beaver triple $([r], [r_i], [c_i])$. Next, u and u_i can be revealed in a single round, and then $[xyz_i w_i]$ is computed in the Beaver form as $[(u-r) \cdot (u_i-r_i)] = u \cdot u_i - u \cdot [r_i] - u_i \cdot [r] + [c_i]$. Therefore, the vectorized four-input DN multiplication Π_{2L-DN+}^{sh} (Protocol 2) can be realized with a single round of online complexity.

Protocol 2: Π_{2L-DN+}^{sh}

Input: $[x], [y], \{[z_i]\}_{i=1}^m$ and $\{[w_i]\}_{i=1}^m$.

Output: $\{[xyz_i w_i]\}_{i=1}^m$.

Preprocessing:

1. Invoke $\mathcal{F}_{\text{DoubleRand}}^{\text{sh}}$ to generate $m+1$ pairs of double sharings $([r], [r])$ and $\{([r_i], [r_i])\}_{i=1}^m$.
2. For $1 \leq i \leq m$, all parties invoke $\mathcal{F}_{\text{Mult}}^{\text{sh}}$ on $([r], [r_i])$ to compute $[c_i]$.

Online:

1. Compute $[u] = [x] \cdot [y] + [r]$ and $[u_i] = [z_i] \cdot [w_i] + [r_i]$ for $1 \leq i \leq m$.
2. Reveal $u \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([u])$ and $u_i \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([u_i])$.
3. For $1 \leq i \leq m$, all parties locally compute:
 $[xyz_i w_i] = u \cdot u_i - u \cdot [r_i] - u_i \cdot [r] + [c_i]$.

The online complexity is 1 round with $2(m+1)$ field elements per party. As for the preprocessing phase, all random values can be generated in 2 rounds with $4m+1$ field elements per party (including $m+1$ for $\Pi_{\text{DoubleRand}}^{\text{sh}}$, and $3m$ for $\Pi_{\text{Mult}}^{\text{sh}}$).

C. Block Prefix Multiplication

Block prefix multiplication is defined as the prefix multiplication for a small number of inputs, such as 3 or 4, which can be realized using Π_{2L-DN}^{sh} and Π_{2L-DN+}^{sh} . Specifically, given $[a_1], [a_2], [a_3]$, the three-element prefix multiplication protocol $\Pi_{\text{PreMult3}}^{\text{sh}}$ generates $[a_1], [a_1 a_2], [a_1 a_2 a_3]$. This can be trivially derived from the protocol Π_{2L-DN}^{sh} , where the inputs are $[a_1], [a_2], [a_3]$ and $[a_1 a_2]$ can be computed locally as $u - [r]$.

The four-element prefix multiplication protocol $\Pi_{\text{PreMult4}}^{\text{sh}}$ (Protocol 3) can be constructed by combining $\Pi_{\text{PreMult3}}^{\text{sh}}$ and Π_{2L-DN+}^{sh} in parallel, where $[a_1 a_2]$ and $[a_1 a_2 a_3]$ are computed using $\Pi_{\text{PreMult3}}^{\text{sh}}$, and $[a_1 a_2 a_3 a_4]$ is computed through Π_{2L-DN+}^{sh} . Especially, the resulting protocol requires only 3 reveals.

For $\Pi_{\text{PreMult3}}^{\text{sh}}$, the online complexity is 1 round with 4 field elements per party, and the preprocessing complexity is 1 round with 2 field elements per party. As for $\Pi_{\text{PreMult4}}^{\text{sh}}$, the online complexity is 1 round with 6 field elements per party, and the preprocessing complexity is 2 rounds with 6 field elements per party.

D. Prefix OR Protocol

Our prefix OR protocol $\Pi_{\text{PreOR}}^{\text{sh}}$ follows the method in Protocol 1, with the substitution of our secure prefix multiplication subprotocol for binary inputs. Fig. 2 shows our optimized prefix multiplication construction for 61-bit inputs, where operators $\circ$, $\diamond$, and $\blacksquare$ represent two-element, three-element, and four-element multiplications, respectively.

Protocol 3: $\Pi_{\text{PreMult4}}^{\text{sh}}$

Input: $[a_1], [a_2], [a_3]$ and $[a_4]$.

Output: $[a_1], [a_1 a_2], [a_1 a_2 a_3], [a_1 a_2 a_3 a_4]$.

Preprocessing:

1. Invoke $\mathcal{F}_{\text{DoubleRand}}^{\text{sh}}$ to generate 3 pairs of double sharings $([r], [r]), ([r_1], [r_1])$ and $([r_2], [r_2])$.
2. Invoke $\mathcal{F}_{\text{Mult}}^{\text{sh}}$ on $([r], [r_2])$ to compute $[c]$.

Online:

1. Compute $[u] = [a_1] \cdot [a_2] + [r]$, $[u_1] = [r] \cdot [-a_3] + [r_1]$ and $[u_2] = [a_3] \cdot [a_4] + [r_2]$.
2. Reveal $u \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([u])$, $u_1 \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([u_1])$ and $u_2 \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([u_2])$.
3. Compute $[b] = u_1 - [r_1]$.
4. All parties locally compute:
 $[a_1 a_2] = u - [r]$,
 $[a_1 a_2 a_3] = u \cdot [a_3] + [b]$,
 $[a_1 a_2 a_3 a_4] = u \cdot u_2 - u \cdot [r_2] - u_2 \cdot [r] + [c]$.

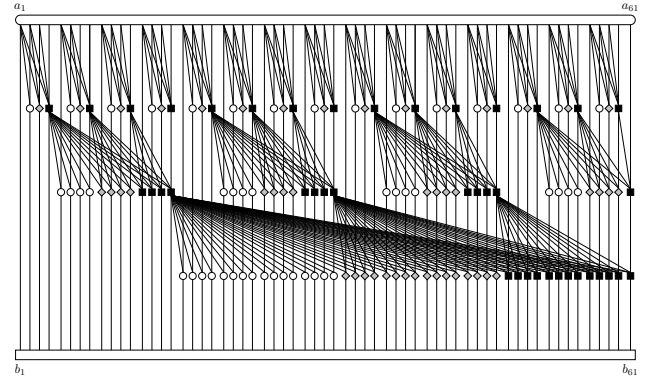


Fig. 2. Prefix multiplication circuit for 61-bit inputs, where $b_i = \prod_{j=1}^i a_j$ for $i \in [1, \ell]$.

In our construction, computations in the first round can be sequentially grouped into sets of four and efficiently executed using our block prefix multiplication protocols. From the second round, consecutive identical operators can be bundled together and computed collectively using vectorized DN multiplication protocols. As a result, by leveraging the protocols in Section IV-B and Section IV-C, we derive our efficient prefix multiplication protocol $\Pi_{\text{PreMult}}^{\text{sh}}$. For ease of exposition, we present the formal description of $\Pi_{\text{PreMult}}^{\text{sh}}$ for the special case where the number of inputs t is a power of 4 in Protocol 4. When it comes to a general bit length, we can substitute some $\Pi_{\text{PreMult4}}^{\text{sh}}$ computations in the first round with $\Pi_{\text{PreMult3}}^{\text{sh}}$ or $\Pi_{\text{Mult}}^{\text{sh}}$, and adjust the vectorization lengths in some Π_{2L-DN}^{sh} or Π_{2L-DN+}^{sh} computations accordingly.

Note that the prefix OR in Protocol 1 only requires one invocation of prefix multiplication, while other steps can be done locally. Therefore, the resulting protocol $\Pi_{\text{PreOR}}^{\text{sh}}$ achieves a better trade-off between online rounds and communication, with only a slight increase in preprocessing complexity. Specifically, the online complexity is $\log_4 \ell$ rounds with $\frac{\ell-4}{3} + \frac{3}{2} \log_4 \ell$ field elements per party, while the PreOpL protocol in [35] requires $\log_2 \ell$ rounds and $\ell \log_2 \ell$ field

Protocol 4: $\Pi_{\text{PreMult}}^{\text{sh}}$

Input: $[a_1], [a_2], \dots, [a_t]$ where $a_i \in \{0, 1\}$.

Output: $[b_1], [b_2], \dots, [b_t]$ where $b_i = \prod_{j=1}^i a_j$.

1. For $1 \leq i \leq t$ set $[a_i^{(0)}] = [a_i]$.
2. For $0 \leq i \leq \frac{t}{4} - 1$
 - a. Compute $\Pi_{\text{PreMult4}}^{\text{sh}}([a_{4i+1}^{(0)}], [a_{4i+2}^{(0)}], [a_{4i+3}^{(0)}], [a_{4i+4}^{(0)}])$
and output $([a_{4i+1}^{(1)}], [a_{4i+2}^{(1)}], [a_{4i+3}^{(1)}], [a_{4i+4}^{(1)}])$.
3. For $2 \leq i \leq \log_4 t$
 - a. For $1 \leq j \leq \frac{t}{4^i}$
 - * Set $k = 4^i \cdot (j-1)$, $q = 4^i \cdot (j-1) + 4^{i-1}$
 $r = 4^i \cdot (j-1) + 2 \cdot 4^{i-1}$, $s = 4^i \cdot (j-1) + 3 \cdot 4^{i-1}$
 - * For $1 \leq l \leq 4^{i-1}$
 - $[a_{k+l}^{(i)}] = [a_{k+l}^{(i-1)}]$
 - $[a_{q+l}^{(i)}] = \Pi_{\text{Mult}}^{\text{sh}}([a_q^{(i-1)}], [a_{q+l}^{(i-1)}])$
 - * Compute $[a_{r+1}^{(i)}], [a_{r+2}^{(i)}], \dots, [a_{r+4^{i-1}}^{(i)}]$ by:
 $\Pi_{\text{2L-DN}}^{\text{sh}}([a_q^{(i-1)}], [a_r^{(i-1)}], \{[a_{r+l}^{(i-1)}]\}_{l=1}^{4^{i-1}})$.
 - * Compute $[a_{s+1}^{(i)}], [a_{s+2}^{(i)}], \dots, [a_{s+4^{i-1}}^{(i)}]$ by:
 $\Pi_{\text{2L-DN}+}^{\text{sh}}([a_q^{(i-1)}], [a_r^{(i-1)}], [a_s^{(i-1)}], \{[a_{s+l}^{(i-1)}]\}_{l=1}^{4^{i-1}})$.
4. Output $([a_1^{(m)}], [a_2^{(m)}], \dots, [a_t^{(m)}])$, where $m = \log_4 t$.

elements per party.

Proposition 1 (Complexity). When $x > 0$, it always holds that $x \log_2 x > \frac{x-4}{3} + \frac{3}{2} \log_4 x$.

Proof. The proof of this proposition is given in Appendix B of supplementary materials. $\square$

Theorem 1. The protocol $\Pi_{\text{PreMult}}^{\text{sh}}$ securely realizes $\mathcal{F}_{\text{PreMult}}^{\text{sh}}$ (see details in [15]) against a static, semi-honest adversary controlling t corrupted parties in the $(\mathcal{F}_{\text{DoubleRand}}^{\text{sh}}, \mathcal{F}_{\text{Mult}}^{\text{sh}})$ -hybrid model with abort.

Proof. The proof of this theorem is given in Appendix B of supplementary materials. $\square$

V. ACHIEVING MALICIOUS SECURITY

Helix uses batch verification techniques to detect any potential malicious behavior during multiplication operations. In particular, after the circuit evaluation step, Helix initially employs compression protocols (Section V-A) to substantially reduce the number of required multiplication verifications. Then, it utilizes the general batch multiplication verification protocol (Section V-B) to verify the compressed multiplication triples. As a result, by integrating the above components, we present a maliciously secure main protocol (Section V-C) for computing arbitrary arithmetic circuits.

A. Lightweight Compression protocol

1) *Intuition:* After carefully analyzing the computational structure of PPML building blocks, we have the following key insight: *the computations in machine learning, particularly neural network inference, exhibit a strong reusability property.* Specifically, matrix multiplication, which is widely used in

fully connected and convolutional layers of NN, contains lots of inner-product operations that share a common vectorized multiplier. For instance, each row of the left matrix is inner-producted with all columns of the right matrix. Likewise, comparison operations used in almost all of the activation functions of NN involve numerous multiplication operations with a common multiplier, as seen in the prefix multiplication circuit in Fig. 2 or the PreOpL protocol in [35]. Motivated by this observation, we abstract the notions of *reusable multiplication triples* and *reusable inner-product triples*.

Definition 1 (Reusable Multiplication Triples). For m triples $(x_1, y_1, z_1), \dots, (x_m, y_m, z_m)$, where $z_i = x_i \cdot y_i$ for all $i \in [1, m]$, if each x_i can be expressed as a public invertible affine transformation of a common value x over $\mathbb{F}_p$, such that $x_i = f_i(x) = a_i \cdot x + b_i$ with $a_i \neq 0$, and each y_i is unique, then these m triples are defined as reusable multiplication triples with a reuse degree of m .

The definition of reusable inner-product triples can be derived straightforwardly from Definition 1 by substituting multiplication triples with inner-product triples. Note that the case where all a_i 's are equal to 1 and all b_i 's are equal to 0 in Definition 1 represents the most typical form of reusable triples in which the triples share a common multiplier.

When existing multiplication verification protocols [17], [18] are applied to verify the correctness of reusable triples, they treat these triples as independent entities, ignoring the inherent interdependence among them [6], [36]. This introduces considerable redundant computations, resulting in substantial verification overhead. To address this, our intuition is to *compress these reusable triples into a single one using a lightweight method*. Taking reusable multiplication triples as an example, since all affine transformations are publicly known and invertible, each x_i can be converted into the common value x using the inverse transformation $f_i^{-1}(x_i) = a_i^{-1}(x_i - b_i)$. Additionally, y_i and z_i can be processed locally to compute $z'_i = a_i^{-1}z_i - a_i^{-1}b_i y_i$, which forms the new triple (x, y_i, z'_i) . Furthermore, we employ the linear combination technique with random coefficients α_i to compress y_i and z'_i , such that $y = \sum_{i=1}^m \alpha_i y_i$ and $z = \sum_{i=1}^m \alpha_i z'_i$. Since all these triples share the common value x , the resulting combined values naturally satisfy the multiplication relationship $z = x \cdot y$.

2) *Protocols:* Our compression protocol for reusable multiplication triples (Π_{ReComp}) is illustrated in Protocol 5. According to the reuse degree m , the number of multiplication triples to be verified can be reduced to $\frac{1}{m}$ of the original amount.

Protocol 5: Π_{ReComp}

Input: $([x_1], [y_1], [z_1]), \dots, ([x_m], [y_m], [z_m])$, and public parameters $\{a_i, b_i\}_{i=1}^m$, where $x_i = a_i \cdot x + b_i$ with $a_i \neq 0$.

Output: $([x], [y], [z])$.

1. For $1 \leq i \leq m$, compute $[z'_i] = a_i^{-1}[z_i] - a_i^{-1}b_i[y_i]$.
2. Invoke $\mathcal{F}_{\text{Coin}}$ to generate m random elements $\alpha_1, \dots, \alpha_m$.
3. Compute $[y] = \sum_{i=1}^m \alpha_i \cdot [y_i]$ and $[z] = \sum_{i=1}^m \alpha_i \cdot [z'_i]$.

The compression protocol for reusable inner-product triples

can be easily realized by substituting inputs with a set of reusable inner-product triples. As for matrix multiplication, we first highlight the inherent *two-dimensional reusability property*. Specifically, for a matrix triple $(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$, where $\mathbf{X} \in \mathbb{F}_p^{m \times n}$, $\mathbf{Y} \in \mathbb{F}_p^{n \times o}$ and $\mathbf{Z} \in \mathbb{F}_p^{m \times o}$, row-wise reusability property exists in $\mathbf{X}$ and column-wise reusability in $\mathbf{Y}$. This means that any row vector in $\mathbf{X}$ forms a set of reusable inner-product triples with all column vectors in $\mathbf{Y}$, while any column vector in $\mathbf{Y}$ and all row vectors in $\mathbf{X}$ are also reusable inner-product triples.

Thus, to compress matrix triples, we extend Π_{ReComp} to Π_{ReComp2D} (Protocol 6) based on a two-stage approach. First, using the row-wise reusability, we linearly combine the m row vectors in $\mathbf{X}$ and $\mathbf{Z}$, separately, with random coefficients $\{\alpha_i\}_{i=1}^m$. This yields an n -dimensional vector $\mathbf{x}$ and an o -dimensional vector $\mathbf{z}$. Next, we exploit the column-wise reusability by linearly combining the o column vectors in $\mathbf{Y}$ and each element in $\mathbf{z}$ with new random coefficients $\{\beta_i\}_{i=1}^o$, resulting in an n -dimensional vector $\mathbf{y}$ and a scalar z . This procedure yields $(\mathbf{x}, \mathbf{y}, z)$, which satisfies the inner-product operation and thus compresses a matrix triple into a single inner-product triple.

Protocol 6: Π_{ReComp2D}

Input: $([\mathbf{X}], [\mathbf{Y}], [\mathbf{Z}])$, where $\mathbf{X} \in \mathbb{F}_p^{m \times n}$, $\mathbf{Y} \in \mathbb{F}_p^{n \times o}$ and $\mathbf{Z} \in \mathbb{F}_p^{m \times o}$.

Output: $([\mathbf{x}], [\mathbf{y}], [z])$ which is a triple of n -dimensional $[\cdot]$ -shared inner product.

1. Invoke $\mathcal{F}_{\text{Coin}}$ to generate $m + o$ random elements $\alpha_1, \dots, \alpha_m$ and $\beta_1, \dots, \beta_o$.
2. For $1 \leq k \leq n$, all parties locally compute:
 - $[\mathbf{x}(k)] = \sum_{i=1}^m \alpha_i \cdot [\mathbf{X}(i, k)]$
 - $[\mathbf{y}(k)] = \sum_{j=1}^o \beta_j \cdot [\mathbf{Y}(k, j)]$
3. Locally compute $[z] = \sum_{i=1}^m (\sum_{j=1}^o \alpha_i \cdot \beta_j \cdot [\mathbf{Z}(i, j)])$.

For the given matrix triple $(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$, the naive approach requires verification of mo inner-product triples of length n . However, after compression through our Π_{ReComp2D} , only a single n -dimensional inner-product triple is required, reducing the verification count to $\frac{1}{mo}$ of the naive requirement.

3) *Security:* The security of Π_{ReComp} and Π_{ReComp2D} are established in Lemma 1 and Lemma 2, respectively. We defer a full security proof of these lemmas to Appendix C of supplementary materials.

Lemma 1. *If at least one multiplication triple is incorrect, then the compressed multiplication triple output by Π_{ReComp} is correct with probability at most $\frac{1}{p}$.*

Lemma 2. *If the matrix triple is incorrect, then the compressed inner-product triple output by Π_{ReComp2D} is correct with probability less than $\frac{2}{p}$.*

4) *Usage:* Our compression protocols are executed in batches after the circuit evaluation step and before the multiplication verification step. As a result, the multiplication verification protocol only needs to perform batch verification on the compressed triples. For more details, refer to Section

V-C. Note that when using the batch strategy above, the random coefficients used in compression protocols can be reused across separate compression instances for both reusable multiplication triples and matrix triples without compromising security. Moreover, our compression protocols are *verification-method independent*, meaning that they can be seamlessly integrated with any verification approach to narrow the performance gap between malicious and semi-honest protocols.

B. General Batch Multiplication Verification

In this subsection, we adapt the multiplication verification protocol from [18] to the machine learning setting, which enables it to handle an arbitrary number of multiplication triples and various types of triples, including $[\cdot]$ -shared multiplication triples, inner-product triples, and multiplication triples with public outputs. Similar to [18], our method can be divided into three phases: normalization, dimension-reduction, and randomization.

1) *Normalization:* We replace the de-linearization protocol in [18] with a normalization protocol Π_{Norm} (Protocol 7). This phase transforms the verification of various types of triples into a single check over an inner-product triple. Instead of computing exponentiations, we use $\mathcal{F}_{\text{Coin}}$ to generate multiple independent random elements for securely combining triples. The inner product triple $([\mathbf{x}^*], [\mathbf{y}^*], [z^*])$ is integrated into our protocol by scaling $[\mathbf{x}^*]$ and $[z^*]$ with a random value r^* and appending each $r^*[\mathbf{x}^*(i)]$ to the final output triple. For multiplication triples with public outputs $([x^\zeta], [y^\zeta], z^\zeta)$, we leverage the *linearity* of Shamir secret sharing to add the public $r^\zeta z^\zeta$ to the secret-shared $[z]$, incorporating it into Π_{Norm} as well.

Protocol 7: Π_{Norm}

Input: Let $N = m_1 + \sum_{i=1}^{m_2} n_i + m_3$, $N' = m_1 + m_2 + m_3$.

- m_1 pairs of $[\cdot]$ -shared multiplication: $\{([\mathbf{x}_i], [\mathbf{y}_i], [\mathbf{z}_i])\}_{i=1}^{m_1}$
- m_2 pairs of $[\cdot]$ -shared inner product: $\{([\mathbf{x}_i^*], [\mathbf{y}_i^*], [\mathbf{z}_i^*])\}_{i=1}^{m_2}$ with $\mathbf{x}_i^*, \mathbf{y}_i^*$ are n_i -dimensional vector
- m_3 pairs of $[\cdot]$ -shared multiplication with public output: $\{([\mathbf{x}_i^\zeta], [\mathbf{y}_i^\zeta], z_i^\zeta)\}_{i=1}^{m_3}$

Output: $([\mathbf{a}], [\mathbf{b}], [\mathbf{c}])$ which is a triple of N -dimensional $[\cdot]$ -shared inner product.

1. All parties invoke $\mathcal{F}_{\text{Coin}}$ to generate N' random field elements $\{r_i\}_{i=1}^{m_1}, \{r_i^*\}_{i=1}^{m_2}, \{r_i^\zeta\}_{i=1}^{m_3}$.
2. For $1 \leq i \leq m_1$, all parties set $[\mathbf{x}_i] = r_i \cdot [\mathbf{x}_i]$.
3. For $1 \leq i \leq m_2, 1 \leq j \leq n_i$, all parties set $[\mathbf{x}_i^*(j)] = r_i^* \cdot [\mathbf{x}_i^*(j)]$.
4. For $1 \leq i \leq m_3$, all parties set $[\mathbf{x}_i^\zeta] = r_i^\zeta \cdot [\mathbf{x}_i^\zeta]$.
5. Compute $[z] = \sum_{i=1}^{m_1} r_i \cdot [z_i]$, $[z^*] = \sum_{i=1}^{m_2} r_i^* \cdot [z_i^*]$ and $z^\zeta = \sum_{i=1}^{m_3} r_i^\zeta \cdot z_i^\zeta$.
6. Set $[\mathbf{a}] = (\dots, [\mathbf{x}_{i_1}], \dots, [\mathbf{x}_{i_2}^*(j)], \dots, [\mathbf{x}_{i_3}^\zeta], \dots)$,
 $[\mathbf{b}] = (\dots, [\mathbf{y}_{i_1}], \dots, [\mathbf{y}_{i_2}^*(j)], \dots, [\mathbf{y}_{i_3}^\zeta], \dots)$,
 $[\mathbf{c}] = [z] + [z^*] + z^\zeta$, where $i_1 \in [1, m_1], i_2 \in [1, m_2], j \in [1, n_{i_2}], i_3 \in [1, m_3]$.

Lemma 3. *If at least one input tuple is incorrect, then the resulting inner-product triple output by Π_{Norm} is correct with probability at most $\frac{1}{p}$.*

Proof. The proof of this lemma is given in Appendix D of supplementary materials. $\square$

2) *General Dimension-Reduction:* The goal of this phase is to reduce the dimension of the N -dimensional inner-product triple output by Π_{Norm} . Unlike [18], we construct a generalized version that supports arbitrary N , where $N = l_1\tau + l_2$ with l_2 less than the compression parameter τ . The core idea is to divide the input inner-product triple into $\tau + 1$ groups, where the first τ groups are l_1 -dimensional inner-product triples and the last is l_2 -dimensional. Then compress the first τ groups to one l_1 -dimensional inner-product triple according to the Lagrange interpolation method [37]. In other words, it effectively reduces τ verifications to a single verification at a randomly sampled point on the interpolated polynomial. Finally, combine the compressed triple with the last l_2 -dimensional triple using uniformly random numbers, as in Π_{Norm} . As a result, by invoking $\Pi_{\text{DimReduce}}$ once, we can convert the verification of the N -dimensional inner-product triple into a check of an $(l_1 + l_2)$ -dimensional inner-product triple. To further enhance efficiency, we merge several independent operations into the same execution round and design $\Pi_{\text{DimReduce}}$ (Protocol 8) without introducing additional communication rounds.

Protocol 8: $\Pi_{\text{DimReduce}}$

Input: A N -dimensional $[\cdot]$ -shared inner product $([x], [y], [z])$. Let τ denote the compression parameter. Let $N = l_1\tau + l_2$, where $l_2 < \tau$.

Output: A $(l_1 + l_2)$ -dimensional $[\cdot]$ -shared inner product $([a], [b], [c])$.

1. All parties interpret $[x], [y]$ as:
 $[x] = ([a_1], [a_2], \dots, [a_\tau], [u])$
 $[y] = ([b_1], [b_2], \dots, [b_\tau], [v])$,
 where $\{[a_i], [b_i]\}_{i=1}^\tau$ are vectors of dimension l_1 and $[u], [v]$ are vectors of dimension l_2 .
2. Locally compute $[f(\cdot)]$ and $[g(\cdot)]$ by using $\{[a_i]\}_{i=1}^\tau$ and $\{[b_i]\}_{i=1}^\tau$ respectively. $f(\cdot)$ and $g(\cdot)$ are vectors of degree- $(\tau - 1)$ polynomials such that $\forall i \in [1, \tau], f(i) = a_i, g(i) = b_i$.
3. For $\tau + 1 \leq i \leq 2\tau - 1$, all parties locally compute $[a_i] = [f(i)], [b_i] = [g(i)]$.
4. For $1 \leq i \leq 2\tau - 1$, all parties invoke $\Pi_{\text{InnerProd}}$ on $([a_i], [b_i])$ to compute $[c_i]$.
5. Set $[w] = [z] - \sum_{i=1}^\tau [c_i]$.
6. Locally compute $[h(\cdot)]$ by using $\{[c_i]\}_{i=1}^{2\tau-1}$.
7. Invoke $\mathcal{F}_{\text{Coin}}$ to generate two random field elements r and r' .
8. Compute $[a'] = [f(r)], [b'] = [g(r)], [c'] = [h(r)]$.
9. All parties output $[a] = (r' \cdot [a'], [u]), [b] = ([b'], [v])$ and $[c] = r' \cdot [c'] + [w]$.

It is straightforward to observe that the protocol $\Pi_{\text{DimReduce}}$ can be invoked iteratively until the dimension of the resulting inner-product triple is reduced to less than 2τ .

Lemma 4. *If the input inner-product tuple is incorrect, then the resulting inner-product tuple output by $\Pi_{\text{DimReduce}}$ is correct with probability at most $\frac{2\tau-1}{p}$.*

Proof. The proof of this lemma is given in Appendix D of supplementary materials. $\square$

3) *Multiplication Verification with Sub-linear Communication:* After the previous phase, it remains to verify the correctness of an m -dimensional inner-product triple, where $m < 2\tau$. In the randomization phase, this is reduced to the check of a single multiplication triple by adding a random triple, using a compression technique similar to that employed in the dimension-reduction phase. Then following the approach of [18], we construct a general batch multiplication verification protocol, $\Pi_{\text{MultVerify}}$ (Protocol 9), which achieves sub-linear communication complexity. Note that we do not prove full security of $\Pi_{\text{MultVerify}}$. Instead, we directly simulate it in the main protocol.

Protocol 9: $\Pi_{\text{MultVerify}}$

Input: Various types of triples whose format is the same as that in Π_{Norm} .

Output: Accept or abort.

1. *Normalization:* All parties invoke Π_{Norm} on these input triples. Let N -dimensional $([x], [y], [z])$ denote the output.
2. *Dimension-reduction:* All parties invoke $\Pi_{\text{DimReduce}}$ iteratively until the dimension of input $([x], [y], [z])$ is less than 2τ . Suppose the output of this phase is an m -dimensional inner-product triple $([x], [y], [z])$ with $m < 2\tau$.
3. *Randomization:*
 - All parties interpret $[x], [y]$ as:
 $[x] = ([a_1], \dots, [a_m]), [y] = ([b_1], \dots, [b_m])$
 - All parties invoke $\mathcal{F}_{\text{rand}}$ to generate $[a_0]$ and $[b_0]$.
 - Locally compute $[f(\cdot)]$ and $[g(\cdot)]$ by using $\{[a_i]\}_{i=0}^m$ and $\{[b_i]\}_{i=0}^m$ respectively. $f(\cdot)$ and $g(\cdot)$ are degree- m polynomials such that $\forall i \in [0, m], f(i) = a_i, g(i) = b_i$.
 - For $i \in [m + 1, 2m]$, all parties locally compute $[a_i] = [f(i)], [b_i] = [g(i)]$.
 - For $i \in [0, m - 1] \cup [m + 1, 2m]$, all parties invoke Π_{Mult} on $([a_i], [b_i])$ to compute $[c_i]$.
 - Set $[c_m] = [z] - \sum_{i=1}^{m-1} [c_i]$.
 - Locally compute $[h(\cdot)]$ by using $\{[c_i]\}_{i=0}^{2m}$.
 - Invoke $\mathcal{F}_{\text{Coin}}$ to generate a random element r . If $r \in [1, m]$, all parties abort. Otherwise, compute $([a], [b], [c]) = ([f(r)], [g(r)], [h(r)])$.
 - Reveal $a, b, c \leftarrow \Pi_{\text{Reveal-Check}}([a], [b], [c])$. For each party P_i , if $a \cdot b \neq c$, then P_i aborts. Otherwise, outputs accept.

Lemma 5. *If at least one input triple is incorrect, then the protocol $\Pi_{\text{MultVerify}}$ aborts with overwhelming probability.*

Proof. The proof of this lemma is given in Appendix D of supplementary materials. $\square$

Protocol 10: Π_{MPC}

Description: The main protocol of Helix for circuit C .

Initialize: For all $i \in [1, n]$, P_i holds an input $v_i \in \mathbb{F}_p$, a description of the circuit C , and the set $\mathcal{V}_i$ containing all multiplication tuples to be verified.

Preprocessing: All parties invoke $\mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{DoubleRand}}$ to generate a sufficient number of random degree- t sharings, and random double sharings.

Input (Π_{Share}):

1. Parties execute $\Pi_{\text{Share}}^{\text{sh}}$ for every input wire of C . Let $[v_1], \dots, [v_n]$ be the shares generated in this step.
2. Parties invoke $\Pi_{\text{ShareCheck}}$ on $[v_1], \dots, [v_n]$ to check the correctness of these degree- t sharings in batch. If any party does not output accept, the protocol aborts.

Add: Given two shares $[x], [y]$, the parties locally compute $[x] + [y]$.

Multiply (Π_{Mult}): To multiply $[x], [y]$, the parties take an available double sharing $([r], \llbracket r \rrbracket)$. Then, do the following:

1. All parties compute $\llbracket xy + r \rrbracket = [x] \cdot [y] + \llbracket r \rrbracket$ and send their shares to P_{king} .
2. P_{king} reconstructs the value $xy + r$ and sends it back to other parties.
3. All parties compare the hash of the view. If inconsistent, then abort. Otherwise, each party locally computes $[z] = (xy + r) - [r]$ and stores $([x], [y], [z])$ in its $\mathcal{V}$ set for subsequent verification.

Output: Before reconstructing the secrets, the parties must privately verify whether all multiplication tuples stored in the set $\mathcal{V}$ were correct. Let $\{([x_i], [y_i], [z_i])\}_{i=1}^{N_1}$ be the triples stored in $\mathcal{V}$, which is generated by computing multiplication gates. Let $\{([x_j^{(i)}], [y_j^{(i)}], [z_j^{(i)}]), a_j^{(i)}, b_j^{(i)}\}_{j=1}^{m_i}\}_{i=1}^{N_2}$ denote the reusable multiplication triples stored in $\mathcal{V}$, which is produced during circuit evaluation. Here, $\{a_j^{(i)}, b_j^{(i)}\}_{j=1}^{m_i}$ are public parameters that characterize the affine relationships among the m_i triples in the i -th reusable multiplication triple. Let $\{([X_i], [Y_i], [Z_i])\}_{i=1}^{N_3}$ represent the matrix triples stored in $\mathcal{V}$, which is generated by computing matrix multiplications. All parties execute the following steps:

1. Compression Stage
 - For $i \in [1, N_2]$, run Π_{ReComp} on $\{([x_j^{(i)}], [y_j^{(i)}], [z_j^{(i)}]), a_j^{(i)}, b_j^{(i)}\}_{j=1}^{m_i}$. Let $\{([x_i^*], [y_i^*], [z_i^*])\}_{i=1}^{N_2}$ denote the outputs.
 - For $i \in [1, N_3]$, run Π_{ReComp2D} on $([X_i], [Y_i], [Z_i])$. Let $\{[x_i^\zeta], [y_i^\zeta], [z_i^\zeta]\}_{i=1}^{N_3}$ denote the outputs.
2. Verification Stage
 - The parties invoke $\Pi_{\text{MultVerify}}$ on input $\{([x_i], [y_i], [z_i])\}_{i=1}^{N_1}, \{([x_i^*], [y_i^*], [z_i^*])\}_{i=1}^{N_2}$ and $\{[x_i^\zeta], [y_i^\zeta], [z_i^\zeta]\}_{i=1}^{N_3}$. If any party does not output accept, the protocol aborts.
3. Output reconstruction
 - Let $[g]$ denote the sharing of the value on the output wire. All parties reveal $g \leftarrow \Pi_{\text{Reveal-Check}}([g])$. If no party aborts, all parties accept g as the final output.

C. The Main Protocol

We are now ready to present our main protocol Π_{MPC} (Protocol 10). The protocol has a preprocessing phase to generate random materials and an online phase to compute any arithmetic circuit. The set $\mathcal{V}$ is used to collect all multiplication tuples that require verification, which are jointly checked using batch verification techniques in the output phase. There are three types of triples in $\mathcal{V}$: multiplication triples, reusable multiplication triples, and matrix triples, all of which are generated during circuit evaluation. We state the security of the main protocol in Theorem 2. Additionally, we provide an optimization method for view comparisons in Appendix H and an accelerated method [38] for degree- t revealing in Appendix I of supplementary materials.

Theorem 2. *The protocol Π_{MPC} securely realizes the functionality $\mathcal{F}_{\text{MPC}}$ against a static, malicious adversary controlling t corrupted parties in the $(\mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{DoubleRand}}, \mathcal{F}_{\text{Coin}})$ -hybrid model with abort.*

Proof. The proof of this theorem is given in Appendix E of supplementary materials. $\square$

VI. APPLICATIONS TO MACHINE LEARNING

In this section, we present our construction of maliciously secure machine learning building blocks, including matrix multiplication, truncation, and ReLU. Note that, since the verification stage is deferred and performed in batch during the output phase, the actual verification operations are omitted from the protocol description in this section. Instead, storing the multiplication tuples into the set $\mathcal{V}$ implicitly indicates that they will be verified later.

A. Matrix Multiplication

Under Shamir's secret sharing, matrix multiplication can be naturally extended using Π_{Mult} . To ensure correctness under malicious security, we further employ Π_{ReComp2D} and $\Pi_{\text{MultVerify}}$ for efficient verification, resulting in the protocol Π_{MatMult} (Protocol 11).

B. Truncation

We extend the fixed-point multiplication protocol in LXY24 to a maliciously secure version $\Pi_{\text{Fixed-Mult}}$ (Protocol 12). First,

Protocol 11: Π_{MatMult}

Input: $[X]$ and $[Y]$, where $X \in \mathbb{F}_p^{m \times n}$ and $Y \in \mathbb{F}_p^{n \times o}$.

Output: $[Z]$, where $Z = X \cdot Y \in \mathbb{F}_p^{m \times o}$.

Preprocessing:

- Invoke $\mathcal{F}_{\text{DoubleRand}}$ $m \cdot o$ times to obtain $([R], [\mathbf{R}])$, where $R \in \mathbb{F}_p^{m \times o}$.

Online:

1. All parties compute $[X \cdot Y + R] = [X] \cdot [Y] + [\mathbf{R}]$ and send their shares to P_{king} .
2. P_{king} reconstructs the value $X \cdot Y + R$ and sends it back to other parties.
3. All parties compare the hash of the view. If inconsistent, then abort. Otherwise, each party locally computes $[Z] = (X \cdot Y + R) - [R]$ and stores $([X], [Y], [Z])$ in its $\mathcal{V}$ set.

we adapt the semi-honest secure protocol $\Pi_{\text{SolvedBits}}^{\text{sh}}$ for malicious security by incorporating maliciously secure $\mathcal{F}_{\text{Rand}}$ and $\Pi_{\text{MultVerify}}$. Next, we extend $\Pi_{\text{Trunc-Triple}}$ to $\Pi_{\text{Trunc-Quadruple}}$ to generate the truncation quadruple $([r'], [r], [r], [r_{\text{msb}}])$, which can be easily achieved since $[r]$ is already generated by $\Pi_{\text{SolvedBits}}$ in $\Pi_{\text{Trunc-Triple}}$. Finally, based on $\Pi_{\text{Trunc-Quadruple}}$, we can construct $\Pi_{\text{Fixed-Mult}}$ by storing $([a], [b], c - 2^{\ell-2} - [r])$ in the $\mathcal{V}$ set and verifying it in the output phase. Due to space constraints, the formal descriptions of $\Pi_{\text{SolvedBits}}$ and $\Pi_{\text{Trunc-Quadruple}}$ are provided in Appendix F of supplementary materials.

Protocol 12: $\Pi_{\text{Fixed-Mult}}$

Input: $[a]$ and $[b]$.

Output: $[y] \in [\lfloor \frac{a \cdot b}{2^d} \rfloor] + 2E$, where $2E$ is the set $\{0, \pm 1, \pm 2\}$.

Preprocessing:

- Invoke $\Pi_{\text{Trunc-Quadruple}}$ to obtain $([r'], [r], [r], [r_{\text{msb}}])$, where $r' = \lfloor \frac{r}{2^d} \rfloor$.

Online:

1. All parties compute $[c] = [a] \cdot [b] + [r] + 2^{\ell-2}$ and send their shares to P_{king} .
2. P_{king} reconstructs c and sends it back to other parties.
3. All parties compare the hash of the view. If inconsistent, then abort. Otherwise, store $([a], [b], c - 2^{\ell-2} - [r])$ in the $\mathcal{V}$ set.
4. Compute $[e] = (1 - [r_{\text{msb}}]) \cdot c_{\text{msb}}$.
5. Output $[y] = \lfloor \frac{c}{2^d} \rfloor - [r'] + [e] \cdot (2^{\ell-d} - 1) - 2^{\ell-d-2}$.

C. ReLU

As demonstrated in LXY24 [15], the ReLU function can be efficiently realized using the prefix OR protocol, with the majority of the communication cost incurred during the computation of prefix OR. Thus, we focus on presenting the maliciously secure construction of Π_{PreOR} in this subsection, and defer the details of Π_{ReLU} to Appendix F of supplementary materials.

First, we leverage $\mathcal{F}_{\text{DoubleRand}}$ and $\Pi_{\text{MultVerify}}$ to construct maliciously secure versions of the protocols $\Pi_{2\text{L-DN}}$, $\Pi_{2\text{L-DN}+}$, Π_{PreMult3} and Π_{PreMult4} . Taking $\Pi_{2\text{L-DN}+}$ as an example:

- In the preprocessing phase of $\Pi_{2\text{L-DN}+}^{\text{sh}}$ (Protocol 2), we invoke $\mathcal{F}_{\text{DoubleRand}}$, rather than $\mathcal{F}_{\text{DoubleRand}}^{\text{sh}}$, to generate all required double random sharings and use Π_{Mult} to compute $[c_i] = [r \cdot r_i]$. Subsequently, we store the reusable multiplication triples $\{([r], [r_i], [c_i], 1, 0)\}_{i=1}^m$ in the set $\mathcal{V}$.
- In the online phase, after step 2 in Protocol 2, all parties need to compare the hash of their views. The triples $([x], [y], u - [r])$ and $\{([z_i], [w_i], u_i - [r_i])\}_{i=1}^m$ are then stored in $\mathcal{V}$ for subsequent verification.

The other three protocols can be adapted in a similar manner to achieve malicious security.

Then, based on these subprotocols, Π_{PreOR} can be implemented with malicious security. As shown in Fig. 2 and Protocol 4, our protocol Π_{PreOR} exhibits a strong reusability, enabling the use of our Π_{ReComp} protocol to significantly enhance the efficiency of the verification stage. For instance, in Protocol 4, the first three inputs to $\Pi_{2\text{L-DN}+}$ are reused 4^{i-1} times. As a result, the second tuple set $\{([z_i], [w_i], u_i - [r_i])\}_{i=1}^m$, which is inserted into the set $\mathcal{V}$ mentioned above, becomes a collection of reusable multiplication triples in which all z_i 's are equal.

Consequently, after compression by Π_{ReComp} , the complexity of the number of triples to be verified in Π_{PreOR} can be reduced from $O(\ell \log_4 \ell)$ to $O(\ell)$. In particular, the 61-bit protocol requires verification of only 55 triples, indicating a reduction in the number of verified triples to 38% of the original 145 triples in the online phase.

VII. EVALUATION**A. Evaluation Setup**

We implement Helix in C++ using the open-source framework hmpc-public [15]. Consistent with LXY24, our evaluation focuses on scenarios involving 3PC, 7PC, 11PC, 21PC, 31PC, and 63PC. The experiments are conducted on Aliyun ECS using 11 ecs.c6a.8xlarge machines, each equipped with 32 vCPUs and 64 GB of RAM. We consider two network settings: LAN and WAN. These are simulated using the Linux `tc` command, with a bandwidth of 10Gbps (100Mbps) in LAN (WAN), and an average round-trip time of approximately 0.3ms (40ms) in LAN (WAN).

In our implementation, we utilize the Mersenne prime $p = 2^{61} - 1$ for Shamir secret sharing scheme to ensure 40-bit statistical security in the malicious setting. For fixed-point multiplication, the number of fractional bits is set to 13. In terms of the Eigen library [39], we allocate 8 cores to each party to accelerate matrix multiplications.

We assess the secure multi-party inference capabilities of Helix and compare its performance against two baselines: i) the semi-honest scheme LXY24 with the substitution of our secure prefix OR subprotocol. ii) a naive maliciously secure scheme, referred to as "LXY24+", that directly adopts the malicious secure techniques from [18] and does not employ the optimized methods described in Section V. We evaluate

TABLE I
COMPARING THE NUMBER OF VERIFIED TRIPLES BETWEEN LXY24+ AND HELIX.

Protocol	Network-A		Network-B		Network-C	
	Online	Prep.	Online	Prep.	Online	Prep.
LXY24+	157184	43874	268160	183130	2248620	1748070
Helix	15632	36962	62644	153970	590161	1468890
Factor	9.9%	84.2%	23.4%	84.1%	26.2%	84%

3 standard neural networks on the MNIST dataset [40]: a 3-layer DNN derived from SecureML [2] (Network-A), a 3-layer CNN derived from Chameleon [41] (Network-B), and a 4-layer CNN derived from MiniONN [42] (Network-C). The details of neural network structures are presented in Appendix G of supplementary materials.

B. The Number of Verified Multiplication Triples

We first discuss the reduction in the number of multiplication triples to be verified achieved by Helix compared to LXY24+. The experimental results are presented in Table I. These results illustrate that, after applying our compression protocols in Section V-A, the number of verified triples required in the online phase can be reduced to 9.9% for Network A, 23.4% for Network B, and 26.2% for Network C. This significant reduction in the problem scale of multiplication verification highlights the effectiveness of our compression protocols.

C. Running Time in the LAN Setting

In Table II, we evaluate the online and preprocessing time for NN inference in the LAN setting. On average, our maliciously secure Helix is $2.5\times$ slower in the online phase and $1.5\times$ slower in preprocessing compared to the semi-honest secure LXY24. Notably, for 63 parties, our protocol achieves an online (preprocessing) performance of 0.736 (5.512) seconds for Network-C, which is $1.9\times$ ($1.2\times$) slower than LXY24. To the best of our knowledge, this is the first maliciously secure PPML protocol to support 63 parties.

Compared to LXY24+, our protocol achieves a $1.3\times$ to $2.6\times$ improvement in online inference time, with the improvement positively correlated with the parameter size of neural networks. This is because larger network parameters result in more multiplication triples requiring verification, increasing the computational complexity when using the verification protocol in [18]. Consequently, the benefits of our compression protocols become more pronounced as the parameter size grows. Moreover, for preprocessing running time, the improvement is less significant, as the compression factor stabilizes at approximately 84% (compared to a 10% to 30% improvement in the online phase) according to Table I.

D. Running Time in the WAN Setting

Table III presents the online and preprocessing time of the three protocols in the WAN setting. The online (preprocessing) running time of Helix is, on average, $1.5\times$ ($1.6\times$) that of

LXY24. Compared to LXY24+, Helix is slightly faster in both phases, though the difference is less significant than in the LAN setting.

Note that our compression protocols combined with the verification protocol in [18], primarily reduce computational complexity, with limited impact on communication costs. However, in the WAN setting, communication complexity becomes the dominant factor affecting inference performance rather than computational cost. As a result, the performance of Helix and LXY24+ is similar in this scenario. Furthermore, thanks to the sublinear communication of [18], the communication overheads of the maliciously secure Helix and LXY24+ are comparable to those of the semi-honest LXY24. Overall, the three protocols exhibit closer performance in the WAN setting than in the LAN setting.

E. Communication

We report the communication overhead of the three protocols for NN inference in Table IV. Compared to LXY24+, Helix incurs slightly lower communication overhead in the online phase, primarily due to a substantial reduction in the number of multiplication triples requiring verification. This reduction leads to fewer iterations in the verification step and, consequently, lower communication cost. In the preprocessing phase, where the reduction in verified triples is less pronounced, Helix and LXY24+ exhibit nearly identical overhead.

The overhead increment over LXY24 roughly represents the total communication cost introduced by the verification step in the malicious setting. Specifically, for Network-C with 63PC, the communication overhead of the verification step is only 0.057 MB (online) and 1.927 MB (preprocessing), accounting for just 0.02% and 3.6% of Helix's total communication overhead in the respective phases.

VIII. RELATED WORKS

For comparison protocols over fields in the n -party setting, Catrina and de Hoogh [35] achieve both small constant rounds and low communication costs by assuming a large gap between secrets and shares. Damgård *et al.* [33] first study the comparison without the gap requirement, and propose a constant-round prefix OR protocol that is the critical component for realizing gap-free comparison. Nishide and Ohta [34] improve the efficiency of the protocols in [33] without using the bit-decomposition protocol. However, the comparison protocols in [33], [34] both incur impractically high communication costs. The most recent work, Rabbit [43] leverages the OR-version of PreOpL protocol in [35] to bring communication costs to a practical level while depending on logarithmic-round complexity. As a result, for n -party comparison protocols over fields with no gap requirement, existing works exhibit a trade-off between rounds and communication costs, primarily due to the complexity of prefix OR operations.

In the malicious and honest-majority setting, the primary challenge lies in verifying the correctness of multiplications. Lindell and Nof [17] propose a verification protocol using

TABLE II
ONLINE AND PREPROCESSING TIME FOR NN INFERENCE IN THE LAN SETTING. ALL NUMBERS ARE REPORTED IN SECONDS. THE COMPRESSION
PARAMETER τ IN THE MULTIPLICATION VERIFICATION PROTOCOL IS 4.

NN	Protocol	Online						Preprocessing					
		3PC	7PC	11PC	21PC	31PC	63PC	3PC	7PC	11PC	21PC	31PC	63PC
Network-A	LXY24	0.009	0.01	0.011	0.014	0.02	0.048	0.023	0.031	0.041	0.056	0.066	0.179
	LXY24+	0.055	0.057	0.061	0.075	0.094	0.199	0.048	0.059	0.07	0.098	0.127	0.331
	Helix	0.022	0.026	0.029	0.043	0.062	0.152	0.041	0.053	0.065	0.094	0.122	0.315
Network-B	LXY24	0.017	0.018	0.019	0.023	0.029	0.062	0.096	0.134	0.167	0.231	0.265	0.508
	LXY24+	0.092	0.09	0.091	0.104	0.135	0.278	0.169	0.204	0.242	0.319	0.375	0.699
	Helix	0.04	0.044	0.048	0.065	0.086	0.18	0.155	0.187	0.23	0.314	0.373	0.704
Network-C	LXY24	0.121	0.126	0.13	0.167	0.212	0.392	0.924	1.262	1.662	2.219	2.471	4.431
	LXY24+	0.693	0.652	0.638	0.708	0.824	1.403	1.529	1.86	2.244	2.879	3.141	5.468
	Helix	0.262	0.268	0.283	0.345	0.43	0.736	1.347	1.775	2.158	2.786	3.16	5.512

TABLE III
ONLINE AND PREPROCESSING TIME FOR NN INFERENCE IN THE WAN SETTING. ALL NUMBERS ARE REPORTED IN SECONDS. THE COMPRESSION
PARAMETER τ IN THE MULTIPLICATION VERIFICATION PROTOCOL IS 32.

NN	Protocol	Online						Preprocessing					
		3PC	7PC	11PC	21PC	31PC	63PC	3PC	7PC	11PC	21PC	31PC	63PC
Network-A	LXY24	0.563	0.576	0.582	0.636	0.697	0.961	0.386	0.355	0.39	0.427	0.518	1.66
	LXY24+	1.147	1.164	1.171	1.24	1.323	1.928	1.037	1.019	1.054	1.124	1.242	2.403
	Helix	1.013	1.029	1.037	1.102	1.18	1.826	0.813	0.795	0.831	0.897	1.013	2.267
Network-B	LXY24	0.69	0.799	0.765	0.985	1.273	3.529	0.756	0.913	0.901	1.399	1.754	4.252
	LXY24+	1.305	1.409	1.38	1.735	2.051	4.48	1.549	1.642	1.675	2.294	2.906	6.47
	Helix	1.233	1.339	1.313	1.627	1.932	4.451	1.427	1.503	1.508	2.13	2.763	6.533
Network-C	LXY24	3.348	4.72	5.038	7.284	9.547	19.512	6.365	7.999	7.97	12.752	16.757	30.302
	LXY24+	4.684	5.665	6.057	8.585	11.267	24.031	7.726	9.023	9.706	13.517	19.1	33.483
	Helix	4.063	5.352	5.544	7.704	10.004	21.624	7.396	9.477	9.236	13.281	18.742	32.322

TABLE IV
ONLINE AND PREPROCESSING COMMUNICATION COST PER PARTY FOR NN INFERENCE. ALL NUMBERS ARE REPORTED IN MB. THE COMPRESSION
PARAMETER τ IN THE MULTIPLICATION VERIFICATION PROTOCOL IS 4.

NN	Protocol	Online						Preprocessing					
		3PC	7PC	11PC	21PC	31PC	63PC	3PC	7PC	11PC	21PC	31PC	63PC
Network-A	LXY24	0.436	0.561	0.595	0.623	0.633	0.644	0.889	1.136	1.202	1.258	1.277	1.298
	LXY24+	0.44	0.575	0.619	0.673	0.709	0.803	0.913	1.176	1.25	1.314	1.341	1.38
	Helix	0.438	0.565	0.602	0.636	0.652	0.682	0.913	1.176	1.25	1.315	1.342	1.383
Network-B	LXY24	1.84	2.366	2.51	2.629	2.672	2.717	3.721	4.751	5.03	5.261	5.343	5.43
	LXY24+	1.851	2.411	2.591	2.802	2.936	3.275	3.814	4.906	5.205	5.459	5.552	5.665
	Helix	1.842	2.371	2.517	2.643	2.692	2.757	3.814	4.906	5.206	5.459	5.554	5.668
Network-C	LXY24	17.626	22.657	24.029	25.172	25.578	26.01	35.543	45.38	48.039	50.246	51.028	51.861
	LXY24+	17.705	23.049	24.75	26.724	27.964	31.067	36.41	46.815	49.655	52.026	52.872	53.788
	Helix	17.629	22.664	24.039	25.191	25.606	26.067	36.41	46.815	49.655	52.026	52.871	53.788

the triple sacrifice technique [26], which offers low computational complexity while requiring communication linear to the number of multiplication gates. Boneh *et al.* [20] first employ distributed zero-knowledge proofs to achieve sublinear communication at the expense of higher computational complexity. Building on this work, Boyle *et al.* [19] subsequently develop a practical verification protocol based on 3PC replicated secret sharing. Goyal and Song [18] extend the polynomial interpolation-based compression protocol in [37], designing a verification protocol that also achieves sublinear communication while improving practical performance.

In the context of MPC-based PPML, existing works that support malicious security in the honest-majority setting,

mainly focus on 3-4 parties with one corrupt party. ABY3 [5], BLAZE [6], and Falcon [44] employ 3PC replicated secret sharing (RSS) to provide ML inference with malicious security. Works [45], [46], [7], [47], [8] aim to further enhance efficiency in a 4PC setting. More recently, MPClan [48] leverages the RSS scheme to provide maliciously secure ML inference with up to 9 parties. However, RSS-based approaches incur exponential storage overhead, so it is infeasible to scale to a large number of parties, e.g., 63, as us.

IX. CONCLUSION

In this paper, we propose a scalable framework, Helix, for multi-party machine learning inference with malicious

security in the honest majority setting. We design a series of novel protocols to narrow the performance gap between maliciously and semi-honestly secure protocols. Experimental results demonstrate the practicality and scalability of Helix by implementing NN inference with up to 63 parties. Future work could explore migrating our constructions to other maliciously secure frameworks and enhancing efficiency with GPUs.

REFERENCES

- [1] A. C. Yao, “Protocols for secure computations,” in *Proc. 23rd Annu. Symp. Found. Comput. Sci.* IEEE, 1982, pp. 160–164.
- [2] P. Mohassel and Y. Zhang, “SecureML: A system for scalable privacy-preserving machine learning,” in *Proc. IEEE Symp. Secur. Privacy.* IEEE, 2017, pp. 19–38.
- [3] Y. Dong, X. Chen, W. Jing, K. Li, and W. Wang, “Meteor: improved secure 3-party neural network inference with reducing online communication costs,” in *Proc. ACM Web Conf.*, 2023, pp. 2087–2098.
- [4] J.-L. Watson, S. Wagh, and R. A. Popa, “Piranha: A GPU platform for secure computation,” in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 827–844.
- [5] P. Mohassel and P. Rindal, “ABY3: A mixed protocol framework for machine learning,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 35–52.
- [6] A. Patra and A. Suresh, “BLAZE: blazing fast privacy-preserving machine learning,” in *Proc. Symp. Netw. Distrib. Syst. Secur.*, 2020.
- [7] N. Koti, M. Pancholi, A. Patra, and A. Suresh, “SWIFT: Super-fast and robust privacy-preserving machine learning,” in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2651–2668.
- [8] N. Koti, A. Patra, R. Rachuri, and A. Suresh, “Tetrad: Actively secure 4pc for secure training and inference,” in *Proc. Symp. Netw. Distrib. Syst. Secur.*, 2022.
- [9] N. Chandran, D. Gupta, S. L. B. Obbattu, and A. Shah, “Simc: ML inference secure against malicious clients at semi-honest cost,” in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1361–1378.
- [10] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. A. Popa, “Muse: Secure inference resilient to malicious clients,” in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2201–2218.
- [11] G. Xu, X. Han, T. Zhang, S. Xu, J. Ning, X. Huang, H. Li, and R. H. Deng, “Simc 2.0: Improved secure ml inference against malicious clients,” *IEEE Trans. Depend. Sec. Comput.*, 2023.
- [12] I. Damgård, D. Escudero, T. Frederiksen, M. Keller, P. Scholl, and N. Volgushev, “New primitives for actively-secure mpc over rings with applications to private machine learning,” in *Proc. IEEE Symp. Secur. Privacy.* IEEE, 2019, pp. 1102–1120.
- [13] B. Yuan, S. Yang, Y. Zhang, N. Ding, D. Gu, and S.-F. Sun, “{MD-ML}: Super fast {Privacy-Preserving} machine learning for malicious security with a dishonest majority,” in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 2227–2244.
- [14] Y. Zhang, X. Chen, Y. Dong, Q. Zhang, R. Hou, Q. Liu, and X. Chen, “MD-SONIC: Maliciously-secure outsourcing neural network inference with reduced online communication,” *IEEE Trans. Inf. Forensics Security*, 2025.
- [15] F. Liu, X. Xie, and Y. Yu, “Scalable multi-party computation protocols for machine learning in the honest-majority setting,” in *Proc. 33rd USENIX Secur. Symp.*, 2024.
- [16] A. Shamir, “How to share a secret,” *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [17] Y. Lindell and A. Nof, “A framework for constructing fast mpc over arithmetic circuits with malicious adversaries and an honest-majority,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 259–276.
- [18] V. Goyal and Y. Song, “Malicious security comes free in honest-majority mpc,” *Cryptol. ePrint Arch.*, 2020.
- [19] E. Boyle, N. Gilboa, Y. Ishai, and A. Nof, “Practical fully secure three-party computation via sublinear distributed zero-knowledge proofs,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 869–886.
- [20] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai, “Zero-knowledge proofs on secret-shared data via fully linear pcps,” in *Proc. Annu. Int. Cryptol. Conf.* Springer, 2019, pp. 67–97.
- [21] I. Damgård and J. B. Nielsen, “Scalable and unconditionally secure multiparty computation,” in *Proc. Annu. Int. Cryptol. Conf.* Springer, 2007, pp. 572–590.
- [22] D. Beaver, “Efficient multiparty protocols using circuit randomization,” in *Proc. Annu. Int. Cryptol. Conf.* Springer, 1992, pp. 420–432.
- [23] N. Koti, V. B. Kukkala, A. Patra, B. R. Gopal *et al.*, “Find thy neighbourhood: Privacy-preserving local clustering,” *Proc. Privacy Enhancing Technol.*, 2023.
- [24] B. Karmakar, N. Koti, A. Patra, S. Patranabis, P. Paul, and D. Ravi, “Asterisk: Super-fast mpc with a friend,” in *Proc. IEEE Symp. Secur. Privacy.* IEEE, 2024, pp. 542–560.
- [25] A. Patra, T. Schneider, A. Suresh, and H. Yalame, “ABY2. 0: Improved Mixed-Protocol secure Two-Party computation,” in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2165–2182.
- [26] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, “Multiparty computation from somewhat homomorphic encryption,” in *Proc. Annu. Int. Cryptol. Conf.* Springer, 2012, pp. 643–662.
- [27] R. Cramer, I. Damgård, D. Escudero, P. Scholl, and C. Xing, “SPD $\mathbb{Z}_{2^k}$: efficient MPC mod for dishonest majority,” in *Proc. Annu. Int. Cryptol. Conf.* Springer, 2018, pp. 769–798.
- [28] R. Cramer, I. Damgård, and Y. Ishai, “Share conversion, pseudorandom secret-sharing and applications to secure computation,” in *Proc. Theory Cryptogr. Conf.* Springer, 2005, pp. 342–362.
- [29] V. Goyal, H. Li, R. Ostrovsky, A. Polychroniadou, and Y. Song, “Atlas: efficient and scalable mpc in the honest majority setting,” in *Proc. Annu. Int. Cryptol. Conf.* Springer, 2021, pp. 244–274.
- [30] I. Damgård, M. Keller, E. Larraia, V. Pastro, P. Scholl, and N. P. Smart, “Practical covertly secure MPC for dishonest majority—or: breaking the SPDZ limits,” in *Proc. 18th Eur. Symp. Res. Comput. Secur.* Springer, 2013, pp. 1–18.
- [31] R. Canetti, “Universally composable security: A new paradigm for cryptographic protocols,” in *Proc. 42nd IEEE Symp. Found. Comput. Sci.* IEEE, 2001, pp. 136–145.
- [32] J. Bar-Ilan and D. Beaver, “Non-cryptographic fault-tolerant computing in constant number of rounds of interaction,” in *Proc. 8th Annu. ACM Symp. Principles Distrib. Comput.*, 1989, pp. 201–209.
- [33] I. Damgård, M. Fitzi, E. Kiltz, J. B. Nielsen, and T. Toft, “Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation,” in *Proc. Theory Cryptogr. Conf.* Springer, 2006, pp. 285–304.
- [34] T. Nishide and K. Ohta, “Multiparty computation for interval, equality, and comparison without bit-decomposition protocol,” in *Proc. 10th Int. Conf. Pract. Theory Public Key Cryptogr.* Springer, 2007, pp. 343–360.
- [35] O. Catrina and S. De Hoogh, “Improved primitives for secure multiparty integer computation,” in *Proc. 7th Int. Conf. Secur. Cryptogr. Networks.* Springer, 2010, pp. 182–199.
- [36] T. Lu, B. Zhang, L. Li, and K. Ren, “Aegis: A lightning fast privacy-preserving machine learning platform against malicious adversaries,” *Cryptol. ePrint Arch.*, 2023.
- [37] P. S. Nordholt and M. Veeningen, “Minimising communication in honest-majority mpc by batchwise multiplication verification,” in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.* Springer, 2018, pp. 321–339.
- [38] A. Dalskov and D. Escudero, “Honest majority mpc with abort with minimal online communication,” in *Proc. Int. Conf. Cryptol. Inf. Secur. Latin Amer.* Springer, 2021, pp. 453–472.
- [39] G. Guennebaud, B. Jacob *et al.*, “Eigen v3,” 2010. [Online]. Available: <http://eigen.tuxfamily.org>
- [40] Y. LeCun, C. Cortes, and C. J. Burges, “Mnist handwritten digit database,” 2010. Accessed: 2010. [Online]. Available: <http://yann.lecun.com/exdb/mnist>
- [41] M. S. Riazi, C. Weinert, O. Tkachenko, E. M. Songhori, T. Schneider, and F. Koushanfar, “Chameleon: A hybrid secure computation framework for machine learning applications,” in *Proc. Asia Conf. Comput. Commun. Secur.*, 2018, pp. 707–721.
- [42] J. Liu, M. Juuti, Y. Lu, and N. Asokan, “Oblivious neural network predictions via minionn transformations,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [43] E. Makri, D. Rotaru, F. Vercauteren, and S. Wagh, “Rabbit: Efficient comparison for secure multi-party computation,” in *Proc. Int. Conf. Financ. Cryptogr. Data Secur.* Springer, 2021, pp. 249–270.
- [44] S. Wagh, D. Gupta, and N. Chandran, “Falcon: Honest-majority maliciously secure framework for private deep learning,” *Proc. Privacy Enhancing Technol.*, 2021.
- [45] M. Byali, H. Chaudhari, A. Patra, and A. Suresh, “Flash: Fast and robust framework for privacy-preserving machine learning,” *Proc. Privacy Enhancing Technol.*, 2019.
- [46] H. Chaudhari, R. Rachuri, and A. Suresh, “Trident: Efficient 4pc framework for privacy preserving machine learning,” in *Proc. Symp. Netw. Distrib. Syst. Secur.*, 2020.
- [47] A. Dalskov, D. Escudero, and M. Keller, “Fantastic four: Honest-majority four-party secure computation with malicious security,” in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2183–2200.

- [48] N. Koti, S. Patil, A. Patra, and A. Suresh, "Mpcplan: Protocol suite for privacy-conscious computations," *J. Cryptol.*, vol. 36, no. 3, p. 22, 2023.

APPENDIX A FUNCTIONALITIES

In this section, we provide formal definitions of our functionalities.

Functionality $\mathcal{F}_{\text{Rand}}$

On receiving (Rand) from all parties, the functionality waits for a message **Abort** or **Proceed** from the adversary. If it sends **Abort**, the functionality aborts. Otherwise, it proceeds as follows:

1. Receive from the adversary the set of shares $\{r_i\}_{i \in \mathcal{C}}$, where $\mathcal{C}$ denotes the set of all corrupted parties.
2. Sample $r \in \mathbb{F}_p$ uniformly at random.
3. Reconstruct the whole sharing $[r]$ using the secret r and the shares $\{r_i\}_{i \in \mathcal{C}}$, and distribute the corresponding shares to all parties.

Functionality $\mathcal{F}_{\text{DoubleRand}}$

On receiving (DoubleRand) from all parties, the functionality waits for a message **Abort** or **Proceed** from the adversary. If it sends **Abort**, the functionality aborts. Otherwise, it proceeds as follows:

1. Receive from the adversary two sets of shares $\{r_i\}_{i \in \mathcal{C}}$ and $\{r'_i\}_{i \in \mathcal{C}}$, along with an error $\Delta \in \mathbb{F}_p$.
2. Sample $r \in \mathbb{F}_p$ uniformly at random and set $r' = r + \Delta$.
3. For the degree- t sharing, reconstruct the whole sharing $[r]$ using the secret r and the shares $\{r_i\}_{i \in \mathcal{C}}$.
4. For the degree- $2t$ sharing, sample t random elements as the shares of the first t honest parties. Based on the secret r' , the t shares of the first t honest parties, and the shares $\{r'_i\}_{i \in \mathcal{C}}$, reconstruct the whole sharing $\llbracket r' \rrbracket$.
5. Distribute the corresponding shares of $([r], \llbracket r' \rrbracket)$ to all parties.

Functionality $\mathcal{F}_{\text{Coin}}$

On receiving (Coin) from all parties, the functionality samples $r \in \mathbb{F}_p$ uniformly at random. Then it sends r to the adversary. If the adversary replies **Abort**, it aborts. Otherwise, it sends r to all parties.

APPENDIX B

SECURITY PROOFS OF SEMI-HONEST PROTOCOLS

Proof of Proposition 1. Let $f(x) = x \log_2 x - \frac{x-4}{3} - \frac{3}{2}x \log_4 x$. Thus, to prove the above proposition, it suffices to show that $f(x) > 0$ always holds when $x > 0$. By taking the derivative, we obtain $f'(x) = \frac{\ln x + 1}{4 \ln 2} - \frac{1}{3}$, which

Functionality $\mathcal{F}_{\text{MPC}}$

Initialization: On input (Input, $\mathbb{F}_p$) from all parties, the functionality stores the modulus p .

Input: The functionality waits for a message **Abort** or **Proceed** from the adversary. If it sends **Abort**, then abort. Otherwise, on input (Input, P_i , id , x) from P_i and (Input, P_i , id) from all other parties, where id is a fresh identifier, the functionality stores (id, x) .

Add: On input (Add, id_1 , id_2 , id_3) from all parties (where id_1 , id_2 are present in memory), retrieve (id_1, x_1) , (id_2, x_2) and store $(id_3, (x_1 + x_2) \bmod p)$.

Multiply: The functionality waits for a message **Abort** or **Proceed** from the adversary. If it sends **Abort**, then abort. Otherwise, on input (Multiply, id_1 , id_2 , id_3) from all parties (where id_1 , id_2 are present in memory), retrieve (id_1, x_1) , (id_2, x_2) and store $(id_3, (x_1 \cdot x_2) \bmod p)$.

Output: On input (Output, id) from all parties (where id is present in memory), the functionality retrieves (id, y) and sends it to the adversary. Wait for a message **Abort** or **Proceed** from the adversary: if it sends **Abort** then the functionality aborts, otherwise the value y is output to all parties.

is easily shown to be an increasing function for $x > 0$. Through numerical analysis, it can be obtained that $f'(x) = 0$ approximately when $x = 0.928$. This indicates that $f(x)$ attains its minimum when $x = 0.928$. Since the minimum value of $f(x)$ is positive, we conclude that $f(x) > 0$ holds for all $x > 0$. $\square$

Proof of Theorem 1. Since $\Pi_{\text{PreMult}}^{\text{sh}}$ is fundamentally based on the DN multiplication protocol, it avoids the security issues encountered with binary inputs in [15]. Additionally, the associativity of field multiplication allows for a straightforward correctness proof of our prefix multiplication circuit. $\square$

APPENDIX C

SECURITY PROOFS OF COMPRESSION PROTOCOLS

Before delving into the details of our maliciously secure protocols, we first provide an important proposition that will be applied in the proofs of almost all subsequent lemmas.

Proposition 2. Let $\delta_1, \delta_2, \dots, \delta_m \in \mathbb{F}_p$, where not all the δ_i 's are zero, and suppose that each δ_i is independent from the uniform distribution sampling α_j , for any $j \in [1, m]$. Then we have

$$\Pr_{\alpha_1, \dots, \alpha_m \leftarrow \mathbb{F}_p} \left(\sum_{i=1}^m \alpha_i \cdot \delta_i = 0 \right) \leq \frac{1}{p}.$$

Proof. Suppose that $\delta_k \neq 0$. Thus, in this case, $\sum_{i=1}^m \alpha_i \cdot \delta_i = 0$ if and only if

$$\alpha_k = -\delta_k^{-1} \cdot \left(\sum_{i=1, i \neq k}^m \alpha_i \cdot \delta_i \right). \quad (1)$$

Since α_k is randomly sampled and chosen independently of all other values, the probability that Eq. (1) holds is at most $\frac{1}{p}$. $\square$

Proof of Lemma 1. Suppose the adversary introduces incorrect values $z_i = x_i \cdot y_i + e_i$ for all $i \in [1, m]$ and at least one $e_i \in \mathbb{F}_p$ is nonzero. Then we have

$$\begin{aligned} z'_i &= a_i^{-1} z_i - a_i^{-1} b_i y_i = a_i^{-1} (x_i \cdot y_i + e_i) - a_i^{-1} b_i y_i \\ &= a_i^{-1} (x_i - b_i) \cdot y_i + a_i^{-1} e_i = x \cdot y_i + a_i^{-1} e_i \end{aligned}$$

Next,

$$\begin{aligned} z &= \sum_{i=1}^m \alpha_i \cdot z'_i = \sum_{i=1}^m \alpha_i \cdot (x \cdot y_i + a_i^{-1} e_i) \\ &= x \cdot y + \sum_{i=1}^m \alpha_i \cdot a_i^{-1} \cdot e_i. \end{aligned}$$

Since $a_i^{-1} \neq 0$ and α_i 's are randomly sampled by $\mathcal{F}_{\text{Coin}}$, the probability that $z = x \cdot y$ holds is at most $\frac{1}{p}$, as established by Proposition 2. $\square$

Proof of Lemma 2. Suppose the adversary causes incorrect matrix triple $\mathbf{Z} = \mathbf{X} \cdot \mathbf{Y} + \mathbf{E}$ and $\mathbf{E} \in \mathbb{F}_p^{m \times o}$ is not a zero matrix. Then we have

$$\begin{aligned} z &= \sum_{i=1}^m \alpha_i \cdot \left(\sum_{j=1}^o \beta_j \cdot \mathbf{Z}(i, j) \right) \\ &= \mathbf{x} \cdot \mathbf{y} + \sum_{i=1}^m \alpha_i \cdot \left(\sum_{j=1}^o \beta_j \cdot \mathbf{E}(i, j) \right). \end{aligned}$$

Assume that there exists a non-zero element in the k -th row of $\mathbf{E}$. Thus, in this case, $\mathbf{Z} = \mathbf{X} \cdot \mathbf{Y}$ if and only if

$$\alpha_k \cdot \left(\sum_{j=1}^o \beta_j \cdot \mathbf{E}(k, j) \right) = - \sum_{i=1, i \neq k}^m \alpha_i \cdot \left(\sum_{j=1}^o \beta_j \cdot \mathbf{E}(i, j) \right). \quad (2)$$

Our aim is to show that Eq. (2) holds with probability $2/p$. Let $\Delta = \sum_{j=1}^o \beta_j \cdot \mathbf{E}(k, j)$. We have the following cases.

– **Case 1** ($\Delta \neq 0$). Eq. (2) holds if and only if

$$\alpha_k = \Delta^{-1} \cdot \left(- \sum_{i=1, i \neq k}^m \alpha_i \cdot \left(\sum_{j=1}^o \beta_j \cdot \mathbf{E}(i, j) \right) \right).$$

Since α_k is randomly sampled by $\mathcal{F}_{\text{Coin}}$ and chosen independently of all other values, the probability that Eq. (2) holds is at most $1/p$.

– **Case 2** ($\Delta = 0$). In this case, the equality may hold. Nevertheless, the probability that $\Delta = 0$ is at most $1/p$, since β_j is distributed uniformly over $\mathbb{F}_p$ and not known to the adversary before introducing $\mathbf{E}(k, j)$.

In summary, the probability that Eq. (2) holds is at most $\frac{1}{p} + (1 - \frac{1}{p}) \cdot \frac{1}{p} \leq \frac{2}{p}$. $\square$

APPENDIX D

SECURITY PROOFS OF MULTIPLICATION VERIFICATION PROTOCOLS

Proof of Lemma 3. Suppose the adversary causes incorrect values $z_i = x_i \cdot y_i + e_i$, $z_i^* = x_i^* \cdot y_i^* + e_i^*$, $z_i^\zeta = x_i^\zeta \cdot y_i^\zeta + e_i^\zeta$ and at least one of e_i, e_i^*, e_i^ζ is non-zero. Then we have

$$\begin{aligned} c &= \sum_{i=1}^{m_1} r_i \cdot z_i + \sum_{i=1}^{m_2} r_i^* \cdot z_i^* + \sum_{i=1}^{m_3} r_i^\zeta \cdot z_i^\zeta \\ &= \mathbf{a} \cdot \mathbf{b} + \sum_{i=1}^{m_1} r_i e_i + \sum_{i=1}^{m_2} r_i^* e_i^* + \sum_{i=1}^{m_3} r_i^\zeta e_i^\zeta. \end{aligned}$$

Since the three $\sum$ have a symmetric structure, we can uniformly represent them as $\sum_{i=1}^{N'} r_i \cdot e_i$. Therefore, in this case, $c = \mathbf{a} \cdot \mathbf{b}$ if and only if

$$\sum_{i=1}^{N'} r_i \cdot e_i = 0. \quad (3)$$

Since at least one e_i is non-zero and the r_i values are distributed uniformly over $\mathbb{F}_p$, the probability that Eq. (3) holds is at most $\frac{1}{p}$ by Proposition 2. $\square$

Proof of Lemma 4. According to Lemma 9 in [18], at least one tuple among the $\tau + 1$ inner-product tuples, $\{([a_i], [b_i], [c_i])\}_{i=1}^\tau, ([u], [v], [w])$, is incorrect. First, if any of the first τ inner-product tuples are incorrect, then by Lemma 7 in [18], the tuple $([a'], [b'], [c'])$ is also incorrect with probability $1 - \frac{2\tau-2}{p}$. Second, if the first τ tuples are all correct, it must be that $\mathbf{u} \cdot \mathbf{v} \neq w$. Thus, with probability $1 - \frac{2\tau-2}{p}$, either $([a'], [b'], [c'])$ or $([u], [v], [w])$ is incorrect.

Suppose the adversary induces incorrect values $c' = \mathbf{a}' \cdot \mathbf{b}' + e_1$ and $w = \mathbf{u} \cdot \mathbf{v} + e_2$, with at least one $e_i \in \mathbb{F}_p$ not equal to zero. Then we have $c = \mathbf{a} \cdot \mathbf{b} + r' \cdot e_1 + e_2$. In this case, $c = \mathbf{a} \cdot \mathbf{b}$ if and only if $r' \cdot e_1 + e_2 = 0$. Since r' is randomly sampled by $\mathcal{F}_{\text{Coin}}$ and chosen independently of e_1 and e_2 , the probability of this condition holding is at most $\frac{1}{p}$.

In summary, the probability that the resulting inner-product tuple is correct is at most $\frac{2\tau-2}{p} + (1 - \frac{2\tau-2}{p}) \cdot \frac{1}{p} < \frac{2\tau-1}{p}$. $\square$

Proof of Lemma 5. The proof can be easily derived from Lemma 3, Lemma 4, and Lemma 10 in [18]. $\square$

APPENDIX E

SECURITY PROOFS OF THE MAIN PROTOCOL

Proof of Theorem 2. We prove security using the UC framework, and the environment $\mathcal{Z}$ plays the role of both the distinguisher and the adversary. We construct a simulator $\mathcal{S}$ that has access to the ideal functionality $\mathcal{F}_{\text{Rand}}$, $\mathcal{F}_{\text{DoubleRand}}$ and interacts with $\mathcal{Z}$ in such a way that the real interaction and the simulated interaction are indistinguishable to $\mathcal{Z}$. The simulator $\mathcal{S}$ works as follows:

- (1) **Simulating the Initialization and Input phases:** $\mathcal{S}$ emulates the Initialize and Input phases honestly on behalf of the virtual honest parties using dummy inputs. If any abort occurs in step 2 of the Input phase, $\mathcal{S}$ sends Abort to $\mathcal{F}_{\text{MPC}}$. Otherwise, it sends Proceed. Since there are at least $t + 1$ honest parties, $\mathcal{S}$ can extract the input

value v_i of each corrupted party P_i , and then uses these values as input to $\mathcal{F}_{\text{MPC}}$.

- (2) **Simulating the Add phase:** The simulator $\mathcal{S}$ executes the protocol honestly on behalf of the virtual honest parties. Then, it sends Add to $\mathcal{F}_{\text{MPC}}$.
- (3) **Simulating the Multiply phase:** $\mathcal{S}$ runs the protocol honestly on behalf of the honest parties. If any abort occurs in step 3 of the Multiply phase, $\mathcal{S}$ sends Abort to $\mathcal{F}_{\text{MPC}}$. Otherwise, it sends Proceed and Multiply. Since there are at least $t + 1$ honest parties, $\mathcal{S}$ can recover the possible error e such that $z = xy + e$.
- (4) **Simulating the Output phase:** In the Output phase, $\mathcal{S}$ first calls $\mathcal{F}_{\text{MPC}}$ to obtain the correct output y . Next, if there exists any error $e \neq 0$ in the Multiply phase, $\mathcal{S}$ sends Abort to $\mathcal{F}_{\text{MPC}}$. Otherwise, it modifies the shares of the virtual honest parties to ensure that the final output sharing is consistent with the actual value y . Subsequently, $\mathcal{S}$ runs the output reconstruction with the adversary on behalf of the honest parties. If this aborts, then $\mathcal{S}$ sends Abort to $\mathcal{F}_{\text{MPC}}$, otherwise it sends Proceed.

Now we argue that $\mathcal{Z}$ cannot distinguish between the real and ideal executions in both cases. This is clear for the Initialize and Add phases, as they consist only of local computations. As for the Input and Multiply phases, $\mathcal{Z}$ only sees the uniformly random values in both executions, rendering them indistinguishable. In the Output phase, by Lemma 1, 2, and 5, if the compression and verification stages pass, then all multiplication computations are carried out correctly with overwhelming probability, which is indistinguishable from the ideal execution. Next, it is easy to see that if the output reconstruction passes, the final value output by the honest parties is the same in both executions. Therefore, the real interaction and the simulated interaction are indistinguishable to $\mathcal{Z}$. $\square$

APPENDIX F PROTOCOLS

In this section, we provide formal descriptions of our protocols that are not included in the main body of the paper.

APPENDIX G NEURAL NETWORKS

The details of neural network structures are presented in Table V.

APPENDIX H AN OPTIMIZATION METHOD FOR VIEW COMPARISONS

In [17], an optimized strategy for degree- $2t$ revealing is presented, wherein view comparisons are performed in batch after the circuit evaluation step and before the verification step. We observe, however, that if a degree- t revealing occurs within the given circuit, which is common in machine learning tasks, a malicious adversary can launch a differential attack during the next degree- t reveal when view comparisons are conducted only after circuit evaluation.

Protocol $\Pi_{\text{SolvedBits}}$

Input: None.

Output: $([r], [r]_B)$ where $[r]_B$ denotes the bit-decomposition of $[r]$ as $([r_0], \dots, [r_{\ell-1}])$ with each $r_i \in \{0, 1\}$.

1. For $0 \leq i \leq \ell - 1$
 - a. All parties invoke $\mathcal{F}_{\text{Rand}}$ to generate $[a]$.
 - b. Compute $\llbracket a^2 \rrbracket = [a] \cdot [a] + \llbracket 0 \rrbracket$ and send it to P_{king} , where $\llbracket 0 \rrbracket$ can be generated using PRG without interactions.
 - c. P_{king} reconstructs the value a and sends it back to other parties.
 - d. All parties compare the hash of their views. If inconsistent, then abort. Otherwise, each stores $([a], [a], a^2)$ in its $\mathcal{V}$ set.
 - e. If $a^2 = 0$, then repeat the above steps a-d. Otherwise, proceed as below.
 - f. Compute $[c] = (\sqrt{a^2})^{-1} \cdot [a]$.
 - g. Set $[r_i] = 2^{-1} \cdot ([c] + 1)$.
2. Compute $[r] = \sum_{i=0}^{\ell-1} 2^i \cdot [r_i]$.
3. Output $([r], [r]_B)$.

Protocol $\Pi_{\text{Trunc-Quadruple}}$

Input: None.

Output: $([r'], \llbracket r \rrbracket, [r], [r_{\text{msb}}])$, where $r' = \lfloor \frac{r}{2^d} \rfloor$.

1. Invoke $\Pi_{\text{SolvedBits}}$ to obtain $([r], [r]_B)$.
2. Set $[r_{\text{msb}}] = [r_{\ell-1}]$.
3. Compute $[r'] = \sum_{i=d}^{\ell-1} 2^{i-d} \cdot [r_i] + \sum_{i=\ell-d}^{\ell-1} 2^i \cdot [r_{\text{msb}}]$.
4. Compute $\llbracket r \rrbracket = \sum_{i=0}^{\ell-1} 2^i \cdot [r_i] \cdot [r_i] + \llbracket 0 \rrbracket$.
5. Output $([r'], \llbracket r \rrbracket, [r], [r_{\text{msb}}])$.

Protocol $\Pi_{\text{Bitwise-LT}}$

Input: a and $[b]_B$.

Output: $[a < b]$.

The procedure is identical to $\Pi_{\text{Bitwise-LT}}^{\text{sh}}$ in LXY24 [15], with the substitution of our maliciously secure protocol Π_{PreOR} , as it involves only local computations.

Protocol Π_{ReLU}

Input: $[a]$.

Output: $[\text{ReLU}(a)]$.

Preprocessing: Invoke $\Pi_{\text{SolvedBits}}$ to generate $([r], [r]_B)$.

Online:

1. Compute $[y] = [2a + r]$ and reveal $y = \Pi_{\text{Reveal}}([y])$.
2. Compute $[b] = y_{\text{sb}} \oplus [r_0]$.
3. Compute $[c] = \Pi_{\text{Bitwise-LT}}(y, [r]_B)$.
4. Compute $[t] = \Pi_{2\text{L-DN}}([b - c], [b - c], \{[a]\})$.
5. Output $[a] - [t]$.

TABLE V

NEURAL NETWORK ARCHITECTURES USED IN OUR EXPERIMENTS.
 $\text{FC}(m \times n)$ DENOTES A FULLY CONNECTED LAYER THAT MAPS AN INPUT VECTOR OF DIMENSION m TO ONE OF DIMENSION n . $\text{Conv}(f \times f \times o, s, p)$ DENOTES A 2D CONVOLUTIONAL LAYER WITH KERNEL SIZE OF $f \times f$, STRIDE OF s , PADDING OF p , AND o OUTPUT CHANNELS.
 $\text{MAXPOOL}(f \times f, s)$ REPRESENTS A MAX-POOLING LAYER WITH WINDOW SIZE $f \times f$ AND STRIDE s .

Network	Architecture
Network-A	$\text{FC}(784 \times 128) \rightarrow \text{ReLU} \rightarrow \text{FC}(128 \times 128) \rightarrow \text{ReLU} \rightarrow \text{FC}(128 \times 10)$
Network-B	$\text{Conv}(2 \times 2 \times 5, 2, 0) \rightarrow \text{ReLU} \rightarrow \text{FC}(980 \times 100) \rightarrow \text{ReLU} \rightarrow \text{FC}(100 \times 10)$
Network-C	$\text{Conv}(5 \times 5 \times 16, 1, 0) \rightarrow \text{Maxpool}(2 \times 2, 2) \rightarrow \text{ReLU} \rightarrow \text{Conv}(5 \times 5 \times 16, 1, 0) \rightarrow \text{Maxpool}(2 \times 2, 2) \rightarrow \text{ReLU} \rightarrow \text{FC}(256 \times 100) \rightarrow \text{ReLU} \rightarrow \text{FC}(100 \times 10)$

We now analyze this potential attack, similar to the one in [48]. Consider a circuit that initially contains a revealing operation with input $[a]$ or $\llbracket a \rrbracket$. Then, a is used in a local linear operation with other secret values, for example, computing $[c] = a \cdot [b] + [r]$. Finally, the degree- t sharing $[c]$ is revealed. Suppose P_{king} is controlled by a malicious adversary and chooses to send two different values, such as a and $a+e$, where e is known to the adversary, to two honest parties (say P_1 and P_2). Following this, each party performs the local linear operations with $a+e$ computed as $[c] + e \cdot [b]$. Consequently, there are two different secret shares within the network: P_1 holds a share of $[c]$, while P_2 holds a share of $[c] + e \cdot [b]$. In the final phase, both P_1 and P_2 send their shares to P_{king} . Since only $t+1$ shares are needed to reconstruct a degree- t sharing, a malicious adversary controlling t parties can simultaneously reconstruct c and $c+e \cdot b$. Thus, by calculating the difference, the adversary can learn b in clear.

Fortunately, the above differential attack is infeasible in the case of degree- $2t$ revealing when only $2t$ parties send their shares to P_{king} , which is a common approach for optimized performance. Specifically, since revealing a degree- $2t$ sharing requires $2t+1$ shares, P_{king} needs at least $2t+2$ different shares to recover two distinct values. However, only $2t$ shares are received.

In summary, to avoid such differential attacks, the view comparison process can be executed before each degree- t revealing (and also before $\Pi_{\text{Reveal-Check}}$). As for degree- $2t$ revealing, the comparison can be performed using the optimization method proposed in [17].

APPENDIX I

BATCH CHECKING FOR REVEALED DEGREE- t SHARINGS

To reduce the computational complexity of correctness checks in degree- t revealing, we give an accelerated approach that defers the check procedure to be completed in batches. Specifically, we propose Π_{Reveal} following the paradigm presented in [38].

Protocol Π_{Reveal}

Input: $[x_1], [x_2], \dots, [x_m]$.

Online:

1. Compare the hash of the view[†]. If inconsistent, then abort.
2. Reveal $x'_i \leftarrow \Pi_{\text{Reveal}}^{\text{sh}}([x_i])$.

Verification:

1. Invoke $\mathcal{F}_{\text{Coin}}$ to generate m random elements $\alpha_1, \dots, \alpha_m$.
2. The parties locally compute: $[y] = \sum_{i=1}^m \alpha_i \cdot [x_i]$ and $y' = \sum_{i=1}^m \alpha_i \cdot x'_i$. Let $[z] = [y] - y'$.
3. Reveal $z \leftarrow \Pi_{\text{Reveal-Check}}([z])$ with correctness check.
4. If z is not equal to 0 then abort, otherwise continue.

[†] Each party stores a string of its view on all broadcasted values up to now [17].